Syrian Arab Republic Ministry

of Higher Education and Scientific Research

Tartous University

Faculty of Information and Communication Technology Engineering Department of Information Technology – Fifth Year



A project prepared to obtain a degree in Information Technology Engineering entitled:

مشروع أعد لنيل إجازة في هندسة تكنولوجيا المعلومات بعنوان:

BeeHoney: An AI-Driven E-Commerce and Trading Platform

منصة " بي هاني" الرقمية لتجارة وتربية النحل المدعومة بالذكاء الاصطناعي

Student Presentation:

Khder Jaber Ahmed Hasan Mahmoud Yousef Samer Mohamed Dgheem

Hasan Adel Youssef Mohamed Anwar Ramadan

Supervisor:

Dr. Engineer Ragheb Othman Ta'ma

2025-2026

الاهداء

قال الله تعالى في محكم التنزيل:

بِسْمِ ٱللهِ الرَّحْمَنِ الرَّحِيمِ

إِلَّا أَن يَشَآءَ اللهُ نَرْفَعُ دَرَجاتٍ مَّن نَّشَاءُ وَفَوْقَ كُلِّ ذِى عِلْم عَلِيمُ *

وَتَحِيَّتُهُمْ فِيهَا سَلَمُ وَءَاخِرُ دَعْوَنهُمْ أَنِ الْحَمْدُ لِلَّهِ رَبِّ الْعَالَمِينَ **

صدق الله العلي العظيم

---

* سورة يوسف الآية 76 -- ** سورة يونس الآية 10

تُهدَى فَرحَةُ يَومِ التّخَرُج وَضَحِكَاتُهُ إِلَى كُلّ إِنْسَانٍ خَالَط الأَلَمَ مَضَاجعَهُ، تَقْلَتِ الآثقَامُ عَلَى صَدرِهِ، رَاجيَا مِن اللهِ تَعَالَى إبدَالَ أحوَالِهِ، وَصَهْرَ آثقامِهِ، بحَقّ قَولِهِ تَعَالَى: (ق وَالْقُرْآنِ الْمَجِيدِ)

وللحلم بقية .....

والله ولى الامر والتوفيق .....

Eng:khder Jaber Ahmed

2019-2026

الفهرس

الفهرس 2

الملخص 4

الفصل الأول: المقدمة..... 5

1) المقدمة 5

1.1 الخلفية السابقة للمشروع..... 5

1.2 مقدمة عن الفصول..... 8

1.3 أهمية المشروع 9

1.4 الدراسة التحليلية للنظام..... 9

الفصل الثاني: القسم النظري..... 11

2) تعريف تطبيقات الويب..... 11

2.1 الخصائص الأساسية..... 11

2.2 تصنيف التطبيقات من حيث السلوك..... 12

2.3 آلية العمل..... 12

2.4 أهمية تطبيقات الويب..... 13

2.5 مقارنة مختصرة مع أنواع البرمجيات الأخرى..... 15

2.6المخدم زبون في سياق خدمات الويب 15

3) 3.1اللغات المستخدمة..... 16

3.2 أطر العمل الستخدمة..... 17

3.3 المكتبات المستخدمة..... 24

4) الهندسة البرمجية والأنماط المعمارية للنظام 27

4,1 البنية الهيكلية والأنماط المعمارية..... 27

4.2 دورة حياة الطلب والبيانات.....29
4.3 الواجهات والأنظمة المساعدة.....29
4.4 التوسع الهيكلي وأنماط الخدمات 30

الفصل الثالث: القسم العملي.....31

5) 5.1 التصميم الهيكلي والتنفيذي للنظام.....31
5.2 مبررات اختيار نظام.....31
5.3 آلية تصميم وإنشاء المخططات.....32
5.4 القواعد البرمجية للمخططات.....32
5.5 العلاقات بين الكائنات والقيود العلاقية.....32
5.6 إدارة واجهة التكوين وبيئة التشغيل.....32
5.7 مخطط قاعدة البيانات 33
6) الواجهات 34
7) طريقة التشغيل.....50

الفصل الرابع: الخاتمة.....52

8) 8.1 التحديات.....52
8.2التوصيات 54
8,3 الخاتمة 56

المراجع 57

المُلخَص

• يقدم هذا المشروع منصة " Bee honey " كنظام معلومات تجاري متكامل بمفهوم B2C.

• **تضم هذه المنصة :**

1) أنظمة فرعية لأتمتة سلة المشتريات من بيع وشراء منتجات العسل بكافة أنواعه وشراء النحل .
2) إدارة المشتركين الذين هم عبارة عن شركات يريدون شراء منتجات النحل أو العسل ، أو نحالين(عملاء) يريدون شراء هذه المنتجات أو لديهم أسئلة حول كل ما يخص منتجات النحل والعسل تجيبهم عليها المنصة .
3) لوحات تحكم فائقة التقدم للتجهيز والشحن لخدمة المشتركين وتسهيل تعاملهم مع المنصة وفهمهم لما تقدمه.

• **تم تطوير النظام عن طريق لغات :** Laravel 12 و Livewire وTailwind CSS لتقديم واجهات زجاجية تفاعلية ومحسنة الأداء .

• تضمنت هذه المنصة 37 واجهة تفاعلية محسنه ومطورة بتقنيات عالية لتسهيل تعامل المستخدم توفير كامل المتطلبات التي يحتاجها حول النحل والعسل ، تحتوي **ثلاثة واجهات رئيسية :**

1_ الواجهة الأساسية (User) التي تحتوي المنتجات .

2_ واجهة ال (Admin) .

3_ واجهة النحالين (Beekeper) الذين يريدون التعامل مع المنصة .

• أسلوب التطوير الذي قمنا باستخدامه هو أسلوب التطوير الرشيق (Methodology Agile) :

تطوير منصة " Bee Honey " التي تجمع بين تقنيات Laravel, Livewire والذكاء اصطناعي مثل شبكة BERT، يُعد أسلوب Agile هو الخيار الأدق نظراً لطبيعة المشروع المرنة والتي تتطلب فحصاً مستمراً لتفاعل المستخدم والذكاء الاصطناعي.

• تقنية الذكاء الاصطناعي AI التي تمت عن طريق ربط الـ Controller بشبكة BERT العصبونية لمعالجة الاستعلامات الذكية وتحسين بيئة العمل وتقليص وقت الإنتظار عند الرد الفوري على اسئلة المستخدمين...

الفصل الأول

المقدمة

1.الخلفية السابقة للمشروع:

1.1.1 خلفية الدراسة والأهمية التاريخية والبيئية (Background and Significance)

تُعد مهنة تربية النحل (Apiculture) من أعرق المهن الإنسانية التاريخية، حيث تمتد جذورها إلى ما يقارب 2400 عام، مع وجود أدلة وشواهد أثرية تؤكد شيوعها واستخدامها في الحضارات القديمة كالحضارة المصرية وحضارة بلاد الرافدين. لا تقتصر أهمية هذه المهنة على البعد التاريخي فحسب، بل تشكل ركيزة أساسية في التوازن البيئي والأمن الغذائي العالمي؛ حيث يساهم النحل بشكل مباشر في تلقيح المحاصيل الزراعية وزيادة إنتاجيتها، فضلاً عن توفير منتجات حيوية ذات قيمة اقتصادية وطبية عالية كالعسل والشمع.

ومع تنوع نطاق هذه المهنة من الممارسات الفردية (الهواة) إلى الشركات الاستثمارية الضخمة، يواجه النحالون اليوم تحديات بيئية وتجارية جسيمة، تتراوح بين ظاهرة انهيار وتفتت مستعمرات النحل، والتعرض للمبيدات الحشرية، وخسارة الموائل الطبيعية، وصولاً إلى آثار التغير المناخي؛ مما يتطلب إدارة قائمة على معرفة عميقة ببيولوجيا النحل والممارسات البيئية السليمة.

1.1.2 ثورة المعلومات ورقمنة القطاعات الحيوية (The Information (Revolution and Digitalization

يعيش المجتمع العالمي اليوم عصراً يتسم بالتطور المتسارع في مجالات التقنية ونظم المعلومات، مستنداً إلى ما يُعرف بـ "ثورة المعلومات". وقد أدى هذا التدفق المعرفي الرقمي إلى تحويل العالم إلى قرية صغيرة متصلة عبر شبكات واسعة النطاق، مما أتاح الوصول إلى البيانات والمعلومات بشكل أسرع وأكثر كفاءة. ولم يعد أثر هذه الثورة محصوراً في قطاعات الاتصالات والتعليم والطب فحسب، بل امتد ليعيد صياغة القطاعات التقليدية كالزراعة واستدامة البيئة من خلال تحسين الأداء وتوفير فرص تطويرية مبتكرة.

في هذا السياق، تطورت المواقع والمنصات الإلكترونية (E-Platforms) من واجهات معلوماتية جامدة إلى بيئات تفاعلية متكاملة تعزز التواصل المباشر وتلبي الاحتياجات النوعية لفئات مهنية محددة. ومن هنا برزت الحاجة الملحة لرقمنه قطاع تربية النحل عبر هندسة حلول برمجية متطورة تضمن تنظيم هذا القطاع وتأمين متطلبات المربين والمستهلكين على حد سواء.

1.1.3 الأنظمة البرمجية المدمجة والتجارة الإلكترونية المتخصصة (Software (Engineering and E-Commerce Integration

مع زيادة حجم التبادل التجاري والمحتوى المعرفي في قطاع النحالة، أصبح بناء أسواق إلكترونية متخصصة أمراً ضرورياً لإدارة العمليات التجارية وفق نماذج العمل المشتركة (B2B & B2C) بمرونة وشفافية. وتتطلب هذه المنظومة الرقمية هندسة عمليات فوترة وأتمتة مالية دقيقة تتعامل مع توثيق الحركات المالية واسترجاعها بكفاءة من قواعد البيانات الضخمة (مثل نظام MySQL)، مما يضمن حفظ حقوق الأطراف وتسهيل تبادل المحاصيل والمستلزمات.

لتحقيق هذه الأهداف، يتجه مهندسو البرمجيات نحو الاعتماد على بيئات تطوير حديثة وأطر عمل متطورة وعالية الأداء (مثل PHP 8.x وإطار العمل Laravel)، والتي تتيح بناء واجهات مستخدم ديناميكية وتفاعلية تضمن تقديم تجربة مستخدم (UX) متميزة، وتحول البيانات التجارية والمعلوماتية إلى رؤى عملية تدعم اتخاذ القرارات الاستراتيجية والتكتيكية لحماية المناحل وتطويرها.

1.1.4 مشكلة البحث والحل المقترح: منصة BeeHoney (Problem Statement and Proposed Solution)

بناءً على التقاطعات السابقة بين عراقة المهنة والتحديات التي تواجهها من جهة، والفرص المتاحة عبر ثورة تكنولوجيا المعلومات من جهة أخرى، يهدف هذا المشروع إلى تصميم وتطوير منصة "BeeHoney" الإلكترونية المتكاملة.

تأتي المنصة كحل برمجية هندسي يخدم مجتمع النحالين والمستثمرين، من خلال توفير واجهات برمجية متطورة لإدارة قواعد البيانات، وتسهيل العمليات التجارية والمالية الآمنة، وعقد المناقشات التفاعلية المنظمة حول الأسعار والأساليب البيئية. تسعى منصة "BeeHoney" إلى تقديم تجربة رقمية آمنة وبسيطة، تضمن توفير وصول سريع وسهل الاستيعاب للمعلومات المتعلقة بصحة الطوائف وتجارة المنتجات، مما يسهم في سد الفجوة التقنية في هذا القطاع الحيوي

تصنيف البرمجيات الحاسوبية (Classification of Software)

تُقسم البرمجيات في علوم الحاسوب وهندسة البرمجيات إلى أربعة تصنيفات رئيسية بناءً على الوظيفة والهدف من تطويرها:

1. برمجيات النظام (System Software)

هي البرمجيات الأساسية التي تُصمّم لتشغيل العتاد الصلب (Hardware) لجهاز الحاسوب، والتحكم به، وإدارة موارده المختلفة. تعمل هذه البرمجيات كطبقة وسيطة بين العتاد وتطبيقات المستخدم، وتتكفل بإدارة البيانات، والذاكرة، والمعالجة.

أبرز الأمثلة: أنظمة التشغيل الحديثة (Operating Systems) مثل Windows وLinux، بالإضافة إلى الأنظمة القديمة كنظام تشغيل القرص (DOS)، وبرامج تعريف المكونات (Device Drivers).

2. البرمجيات التطبيقية (Application Software)

يُشار إليها اختصاراً بـ "التطبيقات"، وهي برامج يتم تطويرها لمساعدة المستخدم النهائي على تنفيذ مهام محددة وعملية تلبي احتياجاته اليومية أو المهنية. تختلف كل برمجية تطبيقية بحسب الغرض الذي صُممت لأجله.

أبرز الأمثلة: حزم معالجة النصوص (مثل Microsoft Word)، برامج إدارة قواعد البيانات، متصفحات الإنترنت، برامج التصميم الهندسي والجرافيكي، والألعاب الإلكترونية.

3. أدوات البرمجة (Programming Software)

هي بيئات وأدوات برمجية مخصصة لمساعدة المطورين ومهندسي البرمجيات في كتابة، واختبار، وتطوير البرامج والأنظمة الأخرى. توفر هذه الأدوات الدعم اللازم لتحويل الشفرة المصدرية (Source Code) إلى لغة الآلة.

أبرز الأمثلة: المترجمات (Compilers)، أدوات تصحيح الأخطاء (Debuggers)، محررات النصوص البرمجية (Text Editors)، وبيئات التطوير المتكاملة (IDEs).

4. البرمجيات الخبيثة / الضارة (Malicious Software - Malware)

هي برمجيات يتم تطويرها بدافع إلحاق الضرر بأنظمة الحاسوب، أو اختراق الخصوصية، أو تخريب البيانات والبرامج الأخرى دون علم المستخدم أو موافقته.

أبرز الأمثلة: الفيروسات (Viruses)، برمجيات التجسس (Spyware)، وفيروسات الفدية (Ransomware).

1-2-مقدمة عن الفصول:

- الفصل الثاني: القسم النظري ,يحتوي على شرح تطبيقات الويب وخصائصها و اللغات المستخدمة

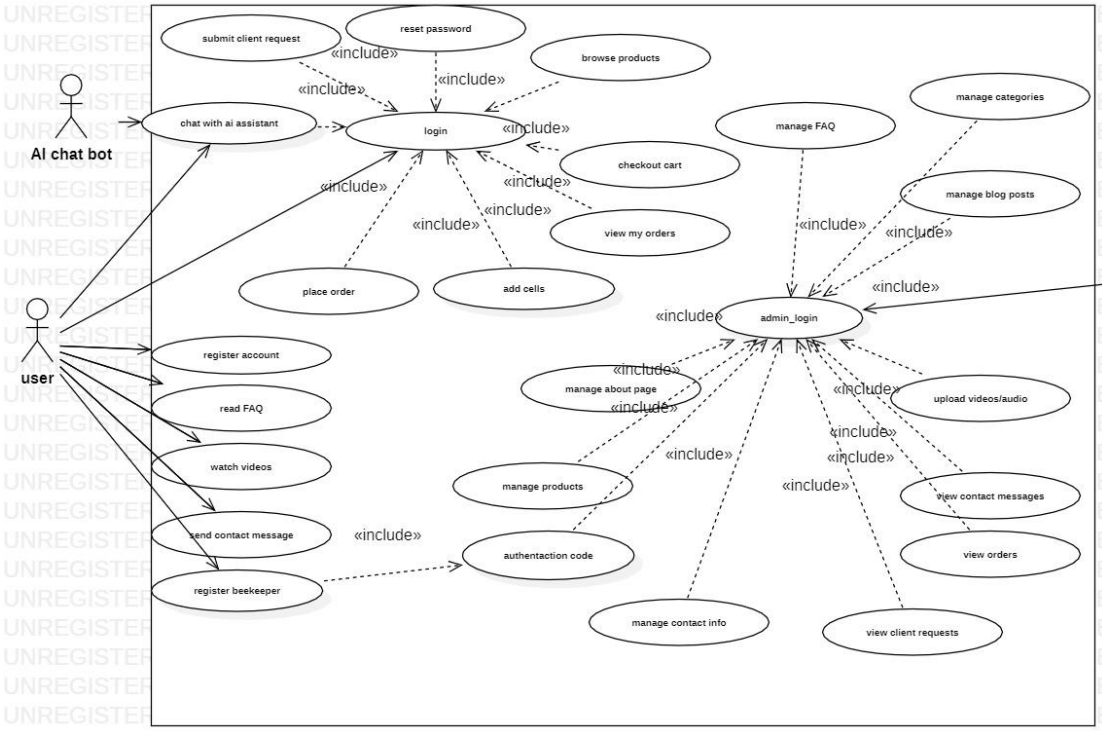
- الفصل الثالث: القسم العملي يحتوي على التطبيق العملي و قواعد البيانات و طريقة التشغيل.

3-1-أهمية المشروع:

تكتسب منصة "BeeHoney" أهمية بالغة بالنظر إلى التقاطعات الحيوية التي تربط بين الحفاظ على قطاع زراعي وتاريخي عريق، وتطبيق أحدث منهجيات هندسة البرمجيات والتجارة الإلكترونية.

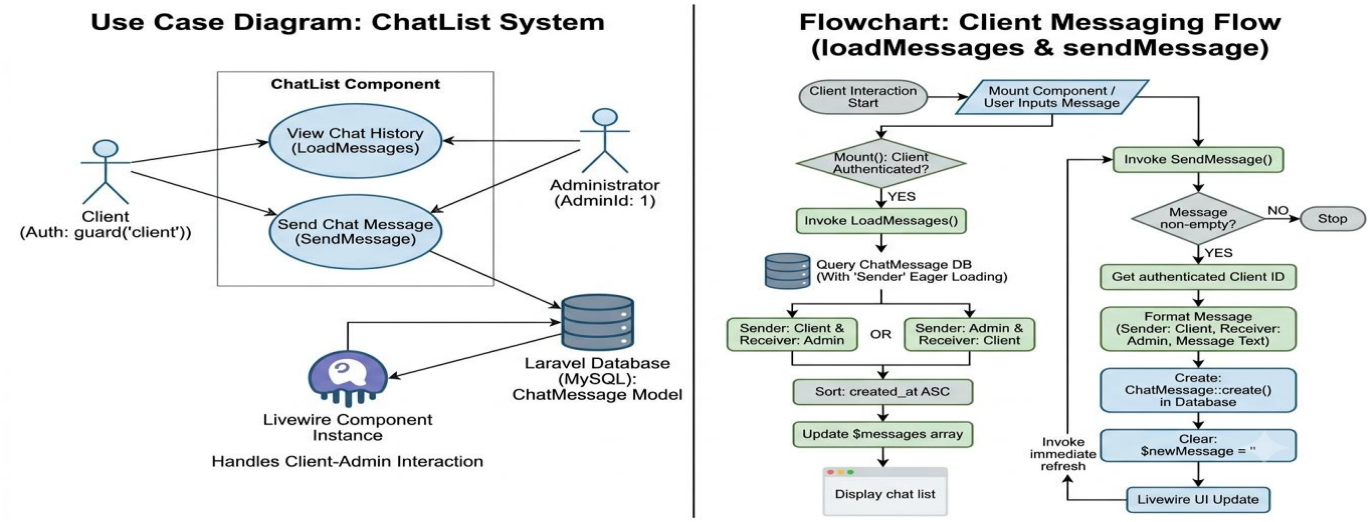
4-1-الدراسة التحليلية للنظام:

مخطط التدفق للنظام:



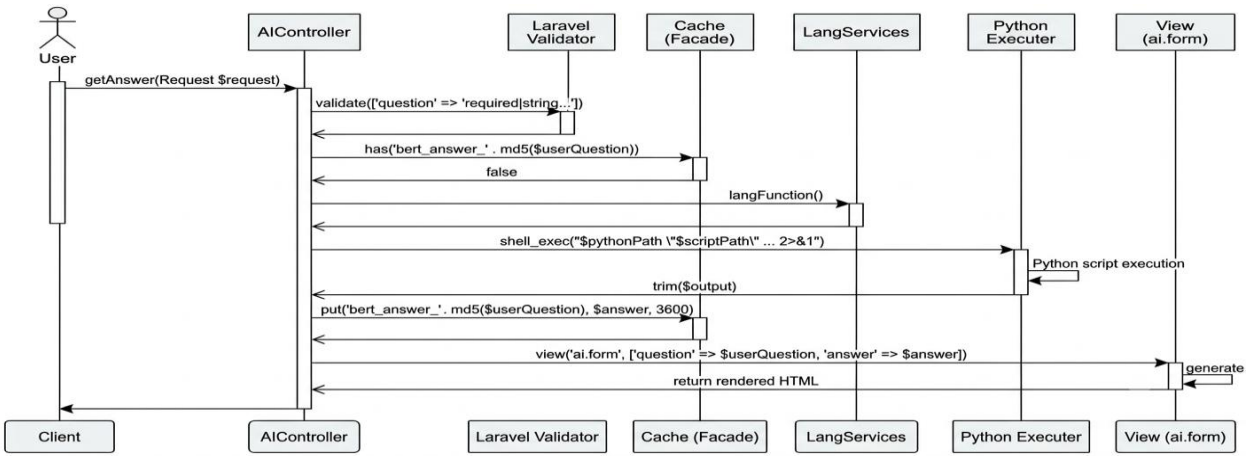
مخطط التدفق:

Chat_diagram:

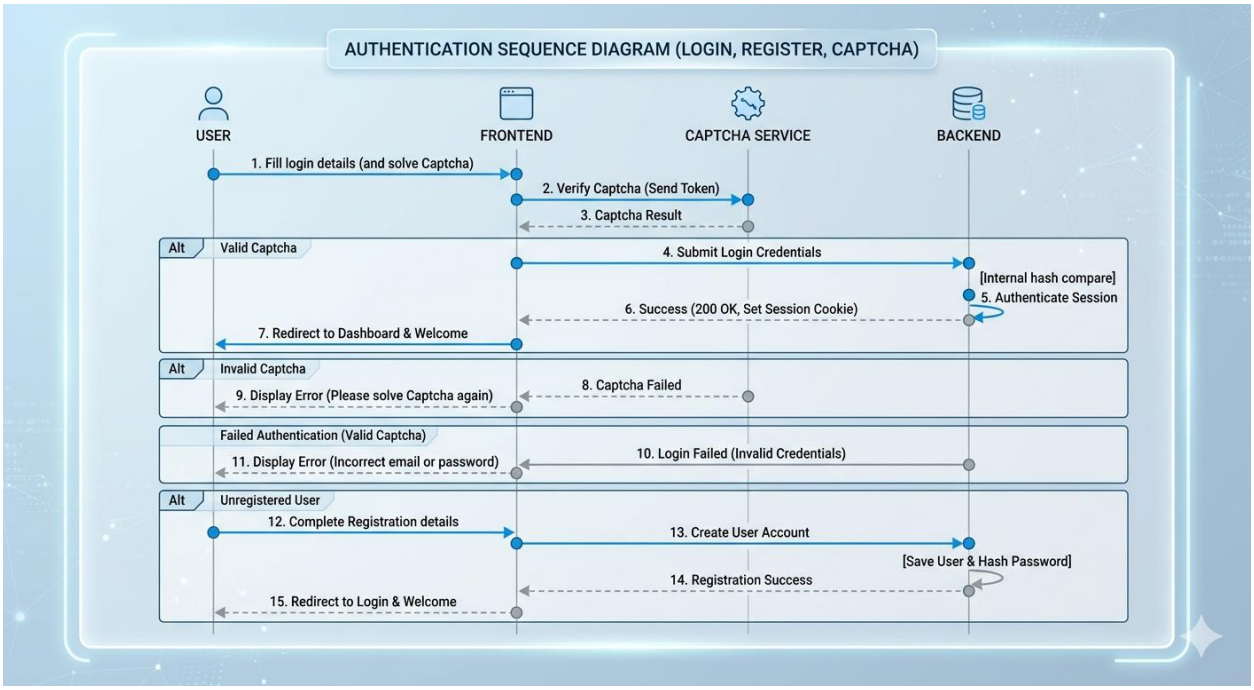


Chatbot Ai:

مخطط التتابع:



نظام المصادقة:



الفصل الثاني

القسم النظري

2-تعريف تطبيق الويب (Web Application)

تطبيق الويب هو برنامج عميل-خادم (Client-Server program) يُخزن ويُنفذ جزئيًا أو كليًا على خادم ويب (Web Server)، ويتفاعل مع المستخدمين عبر متصفح الويب (Web Browser) كواجهة تشغيل. لا يحتاج المستخدم إلى تثبيت البرنامج محليًا، بل يتصل بالخادم عبر شبكة (عادة الإنترنت) باستخدام بروتوكول HTTP/HTTPS.

2-1-الخصائص الأساسية:

a-استقلالية المنصة (Cross-Platform):

يعمل على أي نظام تشغيل يدعم متصفح ويب حديث (Windows, macOS, Linux, Android, iOS).

b-بنية ثلاثية المستويات (Three-Tier Architecture):

1-طبقة العرض (Presentation Layer): المتصفح (HTML, CSS, JavaScript) مسؤول عن واجهة المستخدم.

2-طبقة التطبيق (Application Layer): خادم الويب ولغة البرمجة الخلفية (مثل PHP، Python، Java، C#) وتطبيق منطق الأعمال.

3-طبقة البيانات (Data Layer): خادم قاعدة البيانات (مثل MySQL، PostgreSQL، MongoDB) لتخزين واسترجاع البيانات.

4-عديم الحالة (Stateless) افتراضيًا: يعامل الخادم كل طلب HTTP على أنه مستقل (لا يتذكر الطلبات السابقة). تُستخدم آليات مثل الجلسات (Sessions) والكوكيز (Cookies) أو الرموز (Tokens مثل JWT) للحفاظ على الحالة بين الطلبات.

التحديث المركزي (Centralized Update): أي تحديث أو تصحيح يتم على الخادم ينعكس فورًا على جميع المستخدمين دون تدخل منهم.

2-2تصنيف تطبيقات الويب من حيث السلوك:

1-تطبيقات الصفحة الواحدة (Single Page Application - SPA): يتم تحميل صفحة HTML واحدة، ويتم تحديث المحتوى ديناميكيًا دون إعادة تحميل الصفحة الكاملة (مثل Gmail، Google Maps). تعتمد بكثافة على JavaScript (أطر عمل مثل React، Angular، Vue.js).

2-تطبيقات الويب التقليدية متعددة الصفحات (Multi-Page Application - MPA): كل تفاعل يؤدي إلى طلب صفحة جديدة من الخادم وإعادة تحميل كامل للصفحة (مثل المتاجر الإلكترونية القديمة، المنتديات). تعتمد على PHP، JSP، ASP.NET التقليدي.

2-3-آليات العمل:

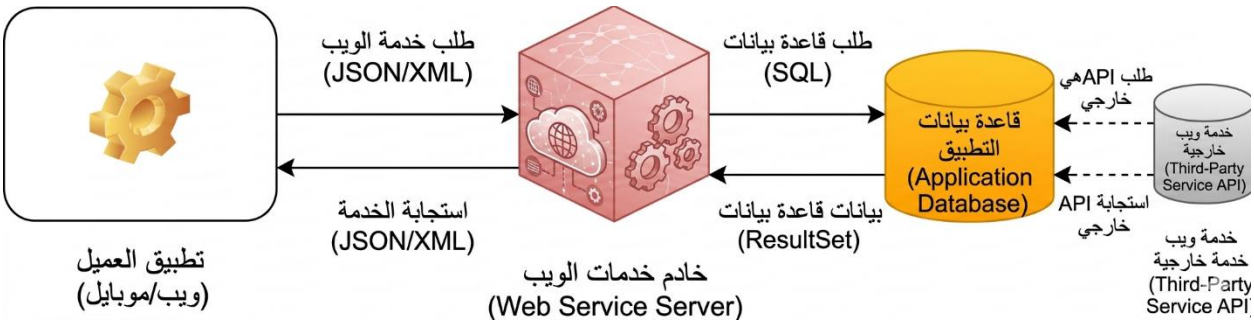
a-يطلب المستخدم عنوان URL (مثلاً https://BeeHoney.com/products).

b-المتصفح يرسل طلب HTTP إلى الخادم.

c-الخادم يعالج الطلب (قد يستعلم قاعدة البيانات، يطبق منطق العمل).

d-الخادم يرسل استجابة HTTP (عادة HTML، CSS، JS، أو بيانات JSON في حالة SPA).

e-المتصفح يعرض واجهة المستخدم.



2-4-أهمية تطبيقات الويب:

1. إمكانية الوصول الشاملة (Universal Accessibility)

أهمية: تزيل حاجز "التثبيت" و"نظام التشغيل".

الشرح: يمكن لأي جهاز مزود بمتصفح (هاتف، حاسوب لوحي، حاسوب مكتبي، حتى أجهزة التلفاز الذكية) الوصول إلى تطبيق الويب. هذا يوسع قاعدة المستخدمين بشكل هائل دون تكاليف تطوير لمنصات **متعددة** (iOS، Android، Windows، macOS). إنه تجسيد لمبدأ "الحوسبة أينما كنت" (Ubiquitous Computing).

2. خفض التكلفة وسهولة الصيانة (Cost Reduction & Centralized Maintenance)

أهمية: تحويل نموذج التكلفة من رأسمالي (شراء تراخيص وتثبيت على كل جهاز) إلى تشغيلي (صيانة خادم واحد).

الشرح: يتم تحديث التطبيق في مكان واحد (الخادم). لا حاجة لتحديث آلاف الأجهزة للمستخدمين. كما لا توجد حاجة لإدارة إصدارات مختلفة لأنظمة التشغيل المختلفة. هذه الميزة هي جوهر نموذج البرمجيات كخدمة (SaaS).

3. الدعم المتأصل للعمل الجماعي والبيانات المركزية (Collaboration & Centralized Data)

أهمية: توفير "مصدر واحد للحقيقة" (Single Source of Truth).

الشرح: البيانات مخزنة مركزياً على الخادم، مما يعني أن جميع المستخدمين يرون نفس البيانات فوراً دون حاجة لمزامنة الملفات (عكس تطبيقات سطح المكتب التقليدية). هذا أساسي لأنظمة إدارة المحتوى (CMS)، وأنظمة إدارة التعلم (LMS)، وأدوات العمل التعاوني مثل Google Docs.

4. قابلية التوسع (Scalability)

أهمية: مواكبة النمو دون إعادة هيكلة جذرية.

الشرح: يمكن توزيع حمل تطبيق الويب على مجموعات من الخوادم (Server Clusters) بسهولة نسبية مقارنة بتطبيقات سطح المكتب. إذا زاد عدد المستخدمين، يمكن إضافة خوادم جديدة (Horizontal Scaling) لاستيعاب الحمل، دون أن يضطر المستخدم النهائي لفعل أي شيء. هذا ما يجعل خدمات مثل Amazon وTwitter قادرة على خدمة مليارات الطلبات.

5. الأمان والتحكم (Security & Control)

أهمية: حماية البيانات الحساسة وإدارة الصلاحيات مركزياً.

الشرح: البيانات لا تُخزن على أجهزة المستخدمين (التي قد تُفقد أو تُسرق)، بل في مراكز بيانات محمية. يتم تطبيق سياسات الأمان والمصادقة (Authentication) والترخيص (Authorization) على الخادم، مما يسمح بإدارة دقيقة لمن يمكنه الوصول إلى ماذا. إضافة إلى أن نسخ الاحتياطي (Backup) يتم مركزياً.

6. سرعة النشر والتكامل (Rapid Deployment & Integration)

أهمية: تقليل الوقت اللازم لطرح ميزة جديدة أو خدمة جديدة في السوق (Time-to-Market).

الشرح: نشر تحديث جديد لتطبيق ويب هو مجرد استبدال ملفات على الخادم. كما أن تطبيقات الويب، من خلال واجهات برمجة التطبيقات (APIs) خاصة واجهات RESTful، يمكنها التكامل بسهولة مع خدمات وتطبيقات أخرى، مما يسمح ببناء أنظمة معقدة من مكونات بسيطة (Service-Oriented Architecture).

7. ثورة في نماذج الأعمال (Business Model Innovation)

أهمية: تمكين نماذج الإيرادات الحديثة.

الشرح: جعلت تطبيقات الويب نماذج الأعمال التالية ممكنة عملياً:

الاشتراك (Subscription): مثل Netflix وSpotify.

الفريميوم (Freemium): خدمة أساسية مجانية، وخدمات متقدمة مدفوعة (مثل Dropbox).

الإعلانات المستهدفة (Targeted Ads): مثل Facebook وGoogle.

2-5-مقارنة مختصرة مع أنواع البرمجيات الأخرى:

موقع ويب ثابت (Static Website)	تطبيق سطح مكتب (Desktop App)	تطبيق ويب	الخاصية
لا حاجة	مطلوب	لا حاجة	**التثبيت**
متصفح فقط	الجهاز المحلي	خادم + متصفح	**التنفيذ**
ثابتة غالباً	محلية أو عبر الإنترنت	ديناميكية (تتغير)	**البيانات**
محدود (روابط، صور)	عالٍ جداً، وصول مباشر للأجهزة	معالجة بيانات وإجراءات معقدة	**التفاعل**
مطلوب لتحميل الصفحات الجديدة	قد لا يكون مطلوباً	مطلوب غالباً	**الاعتماد على الشبكة**

2-6-المخدم زبون /Client - Server/ في سياق خدمات الويب (Web Services):

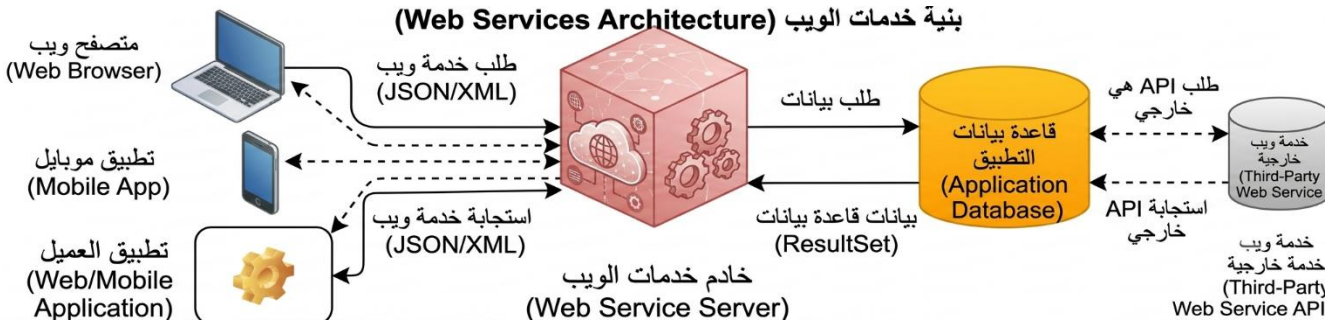
هو نموذج بُنيوي لتطبيق مُوزّع عبر الإنترنت، حيث يجري توزيع المُهام أو الأعمال بين الطرف الذي يُقدّم خدمات الويب (المُسمّى المُخدّم) والطرف الذي يطلبها (المُسمّى العميل).

في هذا النموذج المعياري:

العميل (Client): هو غالباً ما يكون متصفح ويب، تطبيق هاتف ذكي، أو حتى مخدم ويب آخر، يرسل طلبات عبر الإنترنت (Request) باستخدام بروتوكولات قياسية مثل HTTP و JSON/XML. هو لا يشارك موارده ولكنه يطلب الوظائف.

المُخدّم (Server): هو جهاز كمبيوتر قوي ذو مواصفات خاصة يحتوي على ذاكرة كبيرة ومعالج قوي، ويكون مصنوعاً خصيصاً ليعمل كمزود شبكة. هذا المخدم يُشغّل برامج خاصة تستقبل طلبات العميل، وتعالجها (على سبيل المثال، عبر الوصول إلى قاعدة بيانات لتنفيذ استعلام)، ثم تعيد النتيجة كاستجابة (Response) مهيكلة للعميل.

تعتمد شبكات خدمات الويب على هذا التفاعل، حيث تبدأ العملاء بإنشاء قنوات اتصال مع المخدمات لتبادل المعلومات المحددة بشكل آلي ومستقل عن المنصة، مما يسهل دمج التطبيقات المختلفة.



1-3- اللغات المستخدمة وشرح مقتضب عنها:

a-لغات بناء الواجهات وتجربة المستخدم (Front-End)

I-CSS (Cascading Style Sheets)

وظيفتها: لغة تنسيق (وليست لغة برمجة).

شرح: هي المسؤول الأول عن المظهر البصري للموقع. تُستخدم لتحديد الألوان، الخطوط، الأبعاد، وتوزيع العناصر، وجعل الموقع متناسقاً ومناسباً للشاشات المختلفة (الهواتف والحواسب). إذا كان الموقع كائناً حياً، فالـ CSS هي ملابسه ومظهره الخارجي.

II-JS (JavaScript)

وظيفتها: لغة برمجة تفاعلية من جهة العميل (Client-Side).

شرح: هي التي تمنح الموقع الحركة والتفاعلية. تُستخدم لإنشاء عناصر ديناميكية مثل القوائم المنسدلة، أزرار التفاعل، التحديث الفوري للبيانات دون الحاجة لإعادة تحميل الصفحة، والرسوم المتحركة. هي بمثابة الجهاز العصبي الذي يعطي الموقع القدرة على الاستجابة لأفعال المستخدم.

b-لغات معالجة البيانات والعمليات الخلفية (Back-End):

I-PHP (Hypertext Preprocessor)

وظيفتها: لغة برمجة نصية تعمل من جهة المخدم (Server-Side).

شرح: لغة مخصصة ومبنية أساساً لتطوير الويب. تعمل على المخدم لمعالجة البيانات، الاتصال بقواعد البيانات (مثل إرسال طلبات وحفظ بيانات المستخدمين)، وتوليد صفحات الويب بشكل ديناميكي قبل إرسالها للمتصفح. تشغل جزءاً ضخماً من الإنترنت (مثل نظام Laravel).

Python-II

وظيفتها: لغة برمجة عامة الأغراض، عالية المستوى، ومتعددة الاستخدامات.

شرح: تتميز بـ سهولة قراءتها وقوتها. في مجال الويب، تُستخدم كلغة خلفية (Back-End) قوية جداً عبر إطارات عمل مثل (Django وFlask). وما يميزها عن غيرها هو تفوقها أيضاً في مجالات أخرى خارج الويب مثل الذكاء الاصطناعي، تحليل البيانات، والأتمتة.

SQLite-III

على عكس أنظمة قواعد البيانات التقليدية مثل MySQL أو PostgreSQL التي تعتمد على معمارية (Client-Server) وتتطلب خادماً منفصلاً يعمل ويتصل به التطبيق عبر الشبكة، فإن SQLite عبارة عن قاعدة بيانات مدمجة 2.(Embedded Database Engine)

. الخصائص الهيكلية لـ SQLite:

a-بلا خادم (Serverless): لا يحتاج إلى تثبيت برنامج خادم، ولا يحتاج إلى عمليات تشغيل وإيقاف b- (No Background Process). يتفاعل التطبيق مع قاعدة البيانات مباشرة عبر مكتبة برمجية تقرأ وتكتب في ملف.

c-قائمة على ملف واحد (Single File Database): يتم تخزين كامل بنية قاعدة البيانات (الجداول، الفهارس، البيانات) داخل ملف واحد عادي على القرص الصلب.

d-ذاتية الاحتواء (Self-Contained): لا تعتمد على أي حزم خارجية من نظام التشغيل، مما يجعل نقل قاعدة البيانات أمراً في غاية السهولة (مجرد نسخ للملف).

e-التوافق مع معايير ACID: على الرغم من بساطتها، إلا أنها تدعم بالكامل معايير الذرية، الاتساق، العزل، والمتانة ($ACID$ $Properties$) ، مما يضمن سلامة البيانات حتى عند انقطاع الطاقة أو انهيار التطبيق.

3- 2. اطر العمل المستخدمة:

A- Laravel 12

1.ما هو Laravel 12 باختصار؟

هو إطار عمل (Framework) مفتوح المصدر يعتمد على بنية **MVC (Model-View-Controller)**. تم تصميمه ليجعل عملية تطوير تطبيقات الويب أسهل، أسرع، وأكثر أماناً من خلال توفير حلول جاهزة للمهام المتكررة مثل (إرسال البريد، التحقق من الهوية، إدارة قواعد البيانات، وجدولة المهام).

في هذا الإصدار، ركّز فريق لارافيل بشكل أساسي على **الأداء الفائق، الاستقرار البرمجي (Maintenance-focused)، وتحسين تجربة المطورين (DX)** لتجعل التطبيقات الضخمة تعمل بسلاسة هندسية تامة.

2.المكونات الأساسية في) Laravel 12 هيكلية العمل(

يعتمد لارافيل على ثلاثة أجزاء رئيسية لتنظيم الكود:

- **المسارات :(Routing)** بمثابة الخريطة التي توجّه المستخدم؛ عندما يطلب رابطاً معيناً (مثل /profile)، يحدد الرابط أي "Controller" سيقوم بمعالجة هذا الطلب.
- **التحكم :(Controller)** هو عقل التطبيق، يستقبل الطلب من المسار، يعالجه، يتواصل مع قاعدة البيانات، ثم يرسل النتيجة للواجهة.
- **النماذج وقواعد البيانات :(Eloquent ORM)** نظام خارق يتيح لك التعامل مع قاعدة البيانات وكأنها كائنات برمجية (Objects) دون الحاجة لكتابة استعلامات SQL معقدة.
- **محرك القوالب :(Blade Views)** نظام ذكي جداً لدمج كود الـ HTML مع بيانات PHP لتوليد واجهات المستخدم بشكل ديناميكي وسريع.

3.أبرز ميزات وقوة Laravel 12

أ) منظومة الواجهات الأمامية والجيل الجديد(Vite & Inertia)

يأتي Laravel 12 مدعوماً بأداة **Vite** بشكل افتراضي لضغط وتحديث ملفات الـ CSS والـ JavaScript في أجزاء من الثانية أثناء التطوير. كما يدعم بشكل مذهل تقنيات مثل) **Livewire** لكتابة واجهات تفاعلية بالكامل دون مغادرة (PHP أو) **Inertia.js** لربط لارافيل مع React أو Vue بسلاسة خرافية.(

ب) حزم البداية الحديثة (Starter Kits) والاعتماد على Tailwind CSS

عند إنشاء مشروع جديد، يوفر لارافيل حزم جاهزة مثل **Laravel Breeze** أو **Jetstream**. هذه الحزم تمنحك نظام تسجيل دخول، تأكيد حساب، وتغيير كلمة مرور جاهز ومؤمن بالكامل، ومصمم بأحدث تقنيات الـ **Tailwind CSS** مع دعم كامل لمكتبات المكونات العصرية مثل **shadcn/ui** و **Flux**.

ج) الأمان المدمج(Built-in Security)

لارافيل 12 يحميك تلقائياً من أشهر الثغرات الأمنية دون تدخل منك:

- حماية ضد ثغرة حقن قواعد البيانات.(SQL Injection)
- حماية ضد ثغرة تزوير الطلبات عبر المواقع من خلال توفير توكن مدمج تسمى .csrf@
- نظام تشفير قوي جداً لحفظ كلمات المرور.(Hashing)

د) إدارة المهام والاختبارات الـخلفية

- **Queues (الطوابير :(**تتيح لتطبيقك تأجيل المهام الثقيلة (مثل إرسال 10,000 إيميل للمشتركين) لتعمل في الخلفية، حتى لا يضطر المستخدم للانتظار ويتصفح الموقع بسرعة.
- **Pest & PHPUnit:** بيئة مدمجة ومبسطة جداً لكتابة الاختبارات الآلية (Testing) للتأكد من أن كود تطبيقك يعمل بشكل صحيح ولا ينكسر عند التحديث.

Tailwind CSS -B

هو واحد من أشهر وأقوى إطارات العمل (Frameworks) المبنية على لغة CSS في العالم حالياً، وهو الخيار الافتراضي والمدعوم بقوة في بيئات العمل الحديثة مثل Laravel.

شرح مبسط ومباشر لفكرته ومميزاته:

الفكرة الأساسية: ما هو Tailwind CSS؟

على عكس إطارات العمل التقليدية (مثل Bootstrap) التي تعطيك عناصر جاهزة ومحددة الشكل (مثل زر جاهز أو بطاقة جاهزة)، يعتمد Tailwind على مفهوم يُسمى Utility-First (الأدوات أولاً).

هو يقدم لك آلاف الفئات (Classes) الصغيرة والجاهزة، حيث تمثل كل فئة خاصية واحدة محددة في الـ CSS (مثل: لون الخلفية، الهامش، نوع الخط، الأبعاد). أنت تقوم بتركيب هذه الفئات معاً مباشرة داخل كود الـ HTML لتصنع التصميم الخاص بك بدقة وحرية كاملة دون مغادرة ملف الـ HTML.

كيف يكمل Tailwind العمل مع Laravel؟

كما ذكرنا في Laravel، الحزم الافتراضية للواجهات (مثل Breeze و Jetstream ومكتبات المكونات الحديثة) تأتي مصممة بالكامل بـ Tailwind CSS. هذا التكامل يمنحك لوحات تحكم وواجهات تسجيل دخول جاهزة، أنيقة، ومتجاوبة تماماً فور تثبيت المشروع، مع إمكانية تعديل ألوانها وتفاصيلها بمرونة وسهولة عبر فئات تايلوند

3-3-المكتبات المستخدمة:

1. Filament

هو عبارة عن مجموعة من المكونات الجاهزة (Full-stack Components) المصممة خصيصاً لمطوري Laravel لبناء واجهات إدارية متكاملة.

يعتمد Filament في خلفيته على تقنيتين قويتين جداً:

Tailwind CSS: لتنسيق الواجهات وجعلها تبدو عصرية، أنيقة، ومتجاوبة بالكامل مع الهواتف.

Livewire: لتوفير تفاعلية فائقة وسرعة عالية (تحديث البيانات، الفلترة، والبحث) دون الحاجة لكتابة كود JavaScript معقد.

2. المنظومة الرباعية لـ Filament (ماذا يقدم لك؟)

لا يقتصر Filament على لوحات التحكم التقليدية، بل يأتي مقسماً إلى حزم (Packages) يمكنك استخدامها معاً أو بشكل منفصل:

لوحة التحكم (Panel Builder): تمنحك نظام إدارة كامل للموقع (لوحة أدمن) يحتوي على قوائم جانبية، نظام تسجيل دخول، وإدارة صلاحيات المستخدمين فور التثبيت.

منشئ النماذج (Form Builder): يتيح لك بناء نماذج إدخال بيانات (Forms) معقدة جداً (حقول نصية، رفع ملفات، اختيار تواريخ، حقول مكررة) بسطور برمجية بسيطة جداً بلغة PHP.

منشئ الجداول (Table Builder): لعرض البيانات القادمة من قاعدة البيانات في جداول احترافية تدعم: البحث الذكي، الفرز (Sorting)، الفلترة المتقدمة، والتعديل السريع.

الإحصائيات والرسوم البيانية (Widgets): لإضافة مخططات بيانية (Charts) ولوحات رقمية تلخص أداء التطبيق (مثل: عدد المستخدمين الجدد، حجم المبيعات اليومية) بشكل مرئي جميل.

3. المميزات الأساسية

أ) تطوير بسرعة البرق (TALL Stack)

بدل أن تقضي أياماً في ربط واجهة الـ HTML مع الـ Controller وقاعدة البيانات، يتيح لك Filament توليد صفحات كاملة لإدارة أي جدول في قاعدة البيانات (إنشاء، قراءة، تحديث، حذف - CRUD) بأمر برامجي واحد عبر الـ Terminal!

ب) كتابة كود PHP فقط

أكبر ميزة في Filament هي أنك تقوم ببناء وتخصيص لوحة التحكم بالكامل، وتحديد الحقول والجداول وشروط الفلترة باستخدام لغة PHP فقط. لن تحتاج لكتابة كود CSS أو Vue أو React لتعديل الواجهة.

ج) دعم الوضع الداكن تلقائياً (Dark Mode)

تأتي واجهات Filament مدعومة بالوضع الداكن (Dark Mode) بشكل مدمج واحترافي، حيث يمكن للمستخدم التبديل بين الوضعين بضغطة زر دون أي جهد برامجي منك.

د) مرونة التخصيص العالية

على الرغم من أنه يعطيك واجهات جاهزة، إلا أنه لا يقيدك. يمكنك بكل سهولة تعديل أي مكون، إضافة أزرار مخصصة (Actions)، أو حتى بناء مكونات خاصة بك بالكامل إذا تطلب الأمر.

2- Livewire (لايف وايز):

هو إطار عمل (Framework) ثوري ومدهش مخصص لـ Laravel، يحل واحدة من أكبر المشاكل التي تواجه مطوري الويب: كيف تبني واجهات تفاعلية وسريعة دون الحاجة لكتابة أكواد JavaScript معقدة؟

إذا كنت تريد بناء ميزات تفاعلية (مثل: تحديث صفحة المحادثة فوراً، البحث الحي أثناء الكتابة، التحقق من البيانات داخل الحقول بدون إعادة تحميل الصفحة)، فالطريق التقليدي يتطلب منك بناء API بـ Laravel ثم استخدام React أو Vue في الواجهة. هنا يأتي دور Livewire ليغير قواعد اللعبة تماماً.

1. الفكرة الأساسية: كيف يعمل Livewire؟

يعتمد Livewire على جعل الواجهة الأمامية (Front-End) تتحدث مع الخلفية (Back-End) بسلاسة تامة خلف الكواليس.

a-يقوم Livewire برسم الصفحة لأول مرة كقالب Blade عادي (HTML/PHP).

b-عندما يتفاعل المستخدم مع الصفحة (مثل الضغط على زر أو الكتابة في حقل) ، يقوم Livewire بإرسال طلب (AJAX) تلقائياً وفي الخلفية إلى المخدم.

c-يقوم المخدم (Laravel) بإعادة معالجة المكون وتحديث البيانات بلغة PHP.

d-يعيد المخدم كود الـ HTML المُعدّل، ويقوم Livewire بذكاء بتحديث الجزء المتغير فقط في المتصفح دون إعادة تحميل الصفحة كاملة.

2. المكونات الثنائية لـ Livewire (الهيكلية)

عندما ننشئ مكوناً باستخدام Livewire، فإنه ينشئ ملفين يعملان معاً كتوأم:

الملف الأول (Class بلغة PHP): يحتوي على البيانات (Properties) والعمليات (Methods). هذا الملف يعيش على المخدم ويمثل منطق العمل وقاعدة البيانات.

الملف الثاني (View كقالب Blade): يحتوي على كود الـ HTML وتنسيقات الـ CSS (مثل تايلوند). هذا الملف يعيش في المتصفح ويعرض البيانات للمستخدم.

3. المميزات الأساسية:

I-وداعاً للشتات البرمجي: لست بحاجة لتعلم إطارات عمل ضخمة مثل Vue أو React وبناء منظومة منفصلة تماماً (تطوير فصل الواجهات). يمكنك كتابة تطبيق تفاعلي كامل (Single Page Application) باستخدام PHP و Blade فقط.

II-ربط البيانات الحي (Data Binding): يمكنك ربط حقل إدخال بمتغير في كود الـ PHP بمجرد كتابة wire:model="name". بمجرد أن يكتب المستخدم اسمه، يتحدث المتغير في المخدم فوراً!

الأحداث والتفاعلات (Actions): لتنفيذ أي عملية عند الضغط على زر، كل ما تحتاجه هو كتابة wire:click="save"، وسيقوم Livewire باستدعاء الدالة save() الموجودة في ملف الـ PHP مباشرة.

III-التكامل الخارق مع النظام (TALL Stack): يعمل بسلاسة مطلقة مع Laravel، Tailwind CSS، وهو المحرك الأساسي الذي بُنيت عليه أدوات عملاقة مثل Filament PHP لإدارة لوحات التحكم.

IIII-تحميل ذكي وأداء عالٍ: يحتوي على ميزات مدمجة مثل wire: loading لإظهار مؤشر تحميل (Spinner) تلقائياً أثناء معالجة البيانات، وميزة الـ Pagination (تنقل الصفحات) الذكية التي تحديث الجداول دون رمشة عين.

Mpdf-3

هي واحدة من أشهر وأقدم المكتبات المفتوحة المصدر (Open Source) المخصصة للغة PHP، ووظيفتها الأساسية هي توليد ملفات PDF من أكواد HTML و CSS.

إذا كان لديك صفحة ويب، فاتورة، تقرير، أو شهادة مصممة بالـ HTML وتريد تحويلها بضغطة زر إلى ملف PDF متاح للتحميل، فإن mPDF هي الأداة الكلاسيكية للقيام بذلك.

الميزة الأقوى لـ mPDF (دعم اللغة العربية)

أكبر سبب يجعل مطوري الويب العرب يفضلون mPDF على الكثير من المكتبات الأخرى هو دعمها الممتاز والتلقائي للغة العربية والكتابة من اليمين إلى اليسار (RTL).

المكتبات الأخرى غالباً ما تخرج الحروف العربية مقطعة أو مقلوبة وتتطلب إعدادات معقدة (مثل مكتبة FPDF أو TCPDF في حالاتها الافتراضية)، بينما mPDF يتعرف على الخطوط العربية ويقوم بـ "تشبيك" الحروف وضبط الاتجاهات تلقائياً بمجرد تفعيل خاصية الـ RTL.

ميزات مكتبة mPDF الأساسية

a-تحويل HTML/CSS إلى PDF: يمكنك كتابة كود HTML عادي وتنسيقه بالـ CSS، وتستطيع المكتبة قراءته وتحويله إلى صفحات PDF بدقة (يدعم الجداول، الألوان، الخطوط، والحدود).

b-إدارة الخطوط (TrueType Fonts): يمكنك تضمين أي خط مخصص (مثل خطوط Google لـ آريال، تاجويل، وغيرها) داخل ملف الـ PDF لتبدو النصوص بالشكل الذي تريده تماماً.

c-العلامات المائية (Watermarks): تتيح لك إضافة نصوص أو صور كعلامة مائية خلف محتوى الصفحات (مثل كلمة "نسخة تجريبية" أو شعار الشركة).

d-تخصيص الهوامش والترقيم: يمكنك التحكم في أبعاد الصفحة (A4, A5, Letter)، وإضافة رؤوس وتذييلات صفحات (Headers & Footers) ذكية تحتوي على أرقام الصفحات تلقائياً (مثل: صفحة 1 من 5).

e-حماية الملفات: تتيح لك تشفير ملف الـ PDF وضغطه، ومنع المستخدمين من نسخ النصوص أو طباعة الملف إلا بإذن أو بكلمة مرور.

كيف تعمل مع Laravel؟

على الرغم من أن mPDF مكتبة PHP عامة، إلا أن المطورين في بيئة Laravel يستعملونها عادةً عبر حزم جاهزة تسهل التعامل معها، وأشهرها حزمة carlos-meneses/laravel-mpdf.

تصبح طريقة العمل داخل Laravel غاية في السهولة كالتالي:

تصمم الواجهة (الفاتورة مثلاً) كقالب Blade عادي مستخدماً HTML وCSS.

داخل الـ Controller، تمرر البيانات إلى هذا القالب.

تستدعي مكتبة mPDF لتحول هذا القالب البرمجي إلى ملف PDF جاهز للعرض في المتصفح أو التحميل مباشرة.

4- NPM

ببساطة شديدة، تؤدي NPM دورين رئيسيين في عالم البرمجة:

مخزن عالمي ومفتوح (Registry): هو أكبر مستودع برمي في العالم على الإنترنت، يحتوي على ملايين "الحزم" (Packages) والمكتبات الجاهزة التي كتبها مطورون آخرون ومتاحة للجميع مجاناً (مثل مكتبات الرسوم البيانية، أدوات معالجة التواريخ، أو إطارات العمل مثل React و Vue).

أداة لإدارة الحزم (CLI): هي أداة سطر أوامر تنزل تلقائياً عند تثبيت بيئة Node.js على جهازك. تتيح لك كتابة أوامر بسيطة في الـ Terminal لتنزيل، تحديث، أو حذف تلك المكتبات داخل مشروعك بسهولة دون الحاجة لتحميلها يدوياً من المواقع وفك ضغطها.

مكتبةTransformers

قبل ظهور هذه المكتبة، كان هناك فجوة هائلة في الأوساط الأكاديمية بين **البحث العلمي** و**التطبيق العملي** . إذا أراد باحث اختبار فرضية على نموذج لغوي ضخم، كان عليه إعادة كتابة المعادلات الرياضية والشبكات العصبية من الصفر بناءً على الأوراق البحثية المنشورة.

أتت مكتبة Transformersلتضع حلاً فلسفياً يقوم على ثلاثة ركائز أكاديمية:

أ. مأسسة "نقل التعلم(Transfer Learning) "

في الماضي، كان يُعتقد أن الكمبيوتر يجب أن يتعلم اللغة من الصفر لكل مهمة على حدة (مثلاً: نموذج يتعلم العربية ليصنف المشاعر، ونموذج آخر يتعلم العربية ليترجم).

فلسفة المكتبة تقوم على" :**تعلم القواعد العامة أولاً، ثم تخصص لاحقاً** ."النموذج يتم تدريبه على قراءة ومصطلحات ملايين الكتب والمقالات (خسارة ملايين الدولارات ووقت حوسبة هائل)، ثم يأتي الباحث ليأخذ هذا "العقل الجاهز" ويضيف عليه لمسة أخيرة تخص مسألته المحددة.

ب. توحيد واجهات التعامل(Unified Abstraction)

سواء كان النموذج مبنياً بواسطة Google أو Meta أوOpenAI ، وسواء كان مكتوباً بلغة الـ الموترات الخاصة بـ PyTorch أوTensorFlow ، تقدم المكتبة طبقة تجريدية موحدة. هذا التوحيد سمح للمجتمع الأكاديمي بمقارنة النماذج المختلفة تحت نفس المعايير بدقة علمية كاملة.

2.الهندسة العميقة لنموذج BERT والـTransformers

لكي نفهم كيف يعمل BERT نظرياً، يجب أن نفهم المكون الذي بني عليه وهو **المُحوّل (Transformer)**وتحديداً جزء **المُشفر.(Encoder)**

آلية الانتباه الذاتي(Self-Attention Mechanism)

هذا هو الابتكار الأكاديمي الأهم في القرن الحادي والعشرين في مجال الذكاء الاصطناعي. النماذج القديمة (مثل RNNو ($LSTM$كانت تقرأ الكلمات كلمة تلو الأخرى بالترتيب، مما يجعلها تنسى بداية الجملة الطويلة.

أما آلية **الانتباه الذاتي** فتسمح للنموذج بنظر إلى **جميع الكلمات في الجملة دفعة واحدة**، ويقوم بحساب "وزن أو أهمية" كل كلمة بالنسبة للكلمات الأخرى.

مثال توضيحي لفهم السياق:

- الجملة الأولى: "ذهب الرجل إلى **بنك** سوريا لصرف الشيك".
- الجملة الثانية: "جلس الرجل على **بنك** خشبي في الحديقة".

في النماذج القديمة، كلمة" **بنك** "لها نفس التمثيل الرياضي. أما فيBERT ، بفضل الانتباه الذاتي، يربط كلمة "بنك" في الجملة الأولى بـ "سوريا والشيك" فيفهم أنها مؤسسة مالية، ويربطها في الثانية بـ "خشبي والحديقة" فيفهم أنها مقعد.

كيف يتعلم BERT ذاتياً؟(Self-Supervised Learning)

أكبر معضلة في الذكاء الاصطناعي هي الحاجة لبيانات "مُعلمة بشرياً" (أي نصوص مصنفة مسبقاً من البشر)، وهي عملية مكلفة جداً. تم حل هذه المشكلة في BERT عبر استراتيجيتين للتدريب الذاتي دون تدخل بشري:

1. **نمذجة اللغة المقنعة:(Masked Language Modeling - MLM)**

نأخذ نصاً عادياً من ويكيبيديا، ونقوم بإخفاء (قناع ([MASK])لبعض الكلمات عشوائياً (مثلاً 15% من الكلمات). ثم نطلب من الشبكة العصبية التنبؤ بالكلمة المخفية بناءً على السياق القبلي والبعدي معاً.

الجملة الممررة" :تعتبر [MASK]عاصمة الجمهورية لعربيه السوريه <- ".يتعلم النموذج من السياق أن الكلمة هي "دمشق."

2. **التنبؤ بالجملة التالية:(Next Sentence Prediction - NSP)**

نمرر للنموذج جملتين (أ) و (ب)، ونسأله: هل الجملة (ب) منطقياً تأتي بعد الجملة (أ) في السياق الطبيعي؟

يساعد هذا التدريب الأكاديمي النموذج على فهم العلاقات الكلية بين الفقرات وليس فقط الكلمات المفردة.

.3مفهوم الدمج والنظام البيئي مع إطار عمل مثلLaravel

عند الانتقال من البيئة الأكاديمية Python) وبيئات الذكاء الاصطناعي (إلى بيئة تطوير الويب وأنظمة المؤسسات(PHP / Laravel) ، يتغير المفهوم الهندسي من **"تدريب وبناء النماذج "**إلى **"استهلاك واستغلال النماذج."**

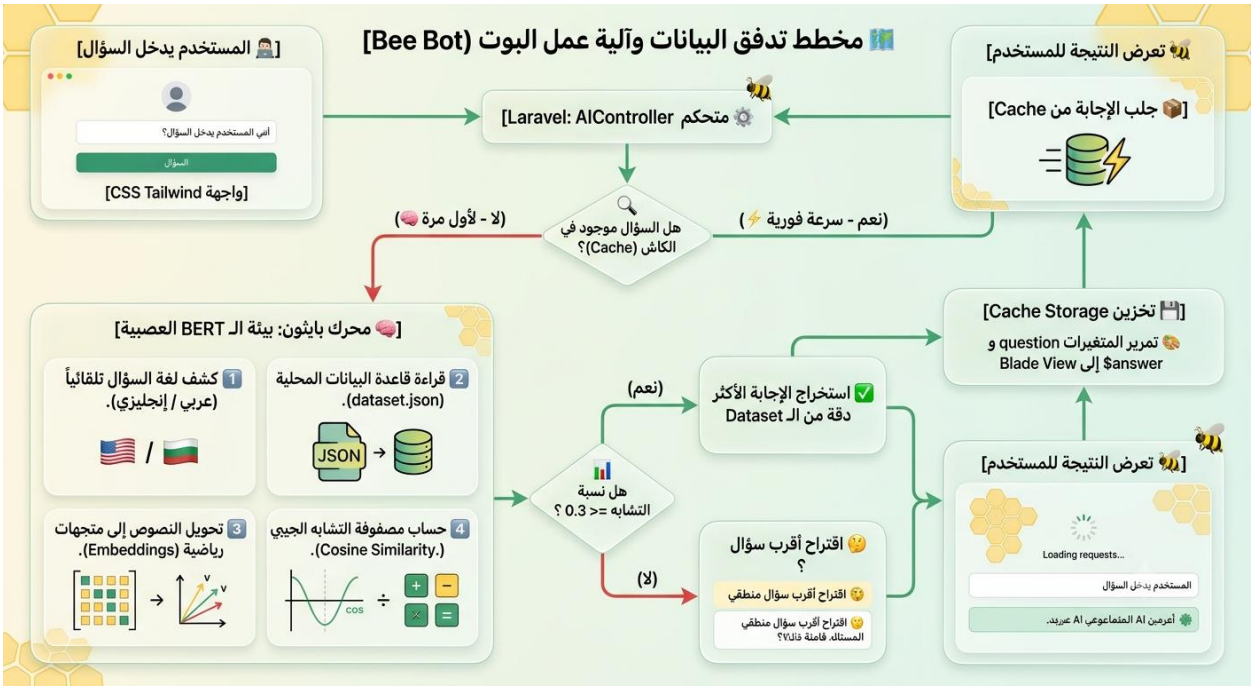
أكاديمياً وهندسياً، لا يتم دمج أوزان BERT الضخمة (والتي قد تبلغ مساحتها مئات الميغابايت أو الجيجابايت) داخل كود الويب الخاص بـ Laravel مباشرة لسببين:

1. **استهلاك الذاكرة (RAM):** لغات الويب مثل PHP مصممة لتستيقظ، تعالج الطلب بسرعة، ثم تموت وتفرغ الذاكرة. بينما نماذج الذكاء الاصطناعي تحتاج للبقاء ثابتة في الذاكرة بشكل دائم.
2. **قوة المعالجة (CPU/GPU):** عمليات المصفوفات الرياضية المعقدة لـ BERT تحتاج لبيئة تدعم الحوسبة المتوازية (مثل سيرفرات بايثون المدعومة بـ كروت شاشة عالية الأداء).

الفلسفة النظرية للربط بين البيئتين:

تعتمد البنية التحتية هنا على فصل المهام(**Decoupling**) ، حيث تلعب البيئتان الأدوار التالية:

- **دور إطار عمل الويب (Laravel):** يمثل "الواجهة الإدارية وبوابة المستخدم". يستقبل النصوص من المستخدمين، يدير قواعد البيانات، يضمن الحماية، ويتحكم في تدفق البيانات الرقمية.
- **دور نموذج الذكاء الاصطناعي (BERT / Hugging Face):** يمثل "المحرك المعرفي المستقل". يقبع في بيئة مخصصة له) سواء سحابية عبر APIs أو محرك محلي منفصل(، ينتظر تزويده بالنص الخام ليعيد تحليله وإرسال النتائج المعرفية (مثل التصنيفات، المستخرجات، أو التقييمات السياقية) إلى Laravel لإظهارها للمستخدم النهائي.



1.مرحلة الاستقبال والتحقق الذكي(Laravel Gateway)

- **البداية :**يدخل المستخدم سؤاله عبر واجهة(Tailwind CSS) ، فيستقبله متحكم لارفيل (AIController).

- **فحص الكاش :(Cache Check)** يقوم النظام بفحص الذاكرة المؤقتة أولاً؛ إذا كان السؤال مكرراً وموجوداً في الكاش، يتم جلب الإجابة فوراً) **سرعة فائقة** (ودون استهلاك موارد السيرفر.

2.مرحلة المعالجة العصبية(Python & BERT)

إذا كان السؤال يُطرح **لأول مرة**، يمرر لارفيل النص بأمان إلى سكربت بايثون لتشغيل محرك الذكاء الاصطناعي:

- **تحديد اللغة :**يكتشف السكربت لغة السؤال تلقائياً (عربي/إنجليزي) ليوجه البحث في الاتجاه الصحيح.
- **قراءة قاعدة البيانات :**يتم استدعاء ملف البيانات المحلي.(dataset. json)
- **التحليل الدلالي :(Embeddings)** يقوم نموذج **BERT**بتحويل سؤال المستخدم وجميع الأسئلة المخزنة إلى متجهات رياضية تفهم معنى الكلمات وسياقها وليس الحروف فقط.

3.مرحلة اتخاذ القرار والرد(Decision Making)

- **حساب التشابه :(Cosine Similarity)** يقيس النموذج نسبة التشابه بين دلالة سؤال المستخدم والأسئلة المخزنة.
- **الشرط الحرج * :(≥ 0.3) إذا تحقق الشرط (نعم) :**يستخرج النظام الإجابة الدقيقة المباشرة المتوافقة مع السؤال.
    - **إذا لم يتحقق (لا) :**لا يتوقف النظام، بل يقترح ذكياً أقرب سؤال منطقي ومسجل لديه لمساعدة المستخدم.

4.مرحلة الحفظ والعرض(Caching & UI)

- **التخزين المؤقت :**تُحفظ الإجابة الناتجة في الكاش لمدة ساعة لضمان السرعة الفورية في المرات القادمة.
- **النهاية :**تُعرض النتيجة بأناقة للمستخدم في واجهة العرض مع تفعيل نظام حماية يمنع تكرار الإرسال أثناء المعالجة

الهندسة البرمجية والأنماط المعمارية للنظام

4_1_: البنية الهيكلية والأنماط المعمارية (Architectural & Design Patterns)

1. الـ Model (النموذج المعماري للبيانات)

التعريف: هو المكون المسؤول عن إدارة الحالة المنطقية للتطبيق (State Management) والتحكم في البيانات وقواعد العمل (Business Rules).

الشرح: يعتمد لارافيل في هذا المكون على نمط تصميمي يُدعى Active Record Pattern عبر حزمة Eloquent. في هذا النمط، يتم إسقاط كل جدول في قاعدة البيانات العلاقية (Relational Database) على فئة برمجية (Class)، ويُمثل كل سطر في الجدول كائنًا (Object) من هذه الفئة، مما يحقق التجريد الكامل لطبقة البيانات (Database Abstraction Layer).

2. الـ Controller (وحدة التحكم والربط)

التعريف: هو المكون الوسيط الذي يربط بين طبقة العرض (View) وطبقة البيانات (Model)، ويُعنى بإدارة تدفق العمليات (Control Flow).

الشرح: يُمثل حرف الـ C في نمط MVC. وظيفته الأكاديمية هي عزل واجهة المستخدم عن منطق تخزين البيانات. يستقبل المدخلات، ويستدعي العمليات المناسبة من النموذج، ثم يوجه النتيجة إلى المكون التالي في دورة حياة الطلب.

3. الـ Route (موجه المسارات الإستراتيجي)

التعريف: هو نظام التوجيه (Routing Engine) الذي يعمل بمثابة نظام خرائط العناوين الموحد (Uniform Resource Identifier - URI Mapping).

الشرح: يقوم على فكرة "التوجيه المبني على القواعد" (Rule-Based Routing). يربط معرفات الموارد الرقمية (URLs) بالمعالجات (Handlers/Controllers) المقابلة لها، وهو الأساس الذي تُبنى عليه معماريات الـ RESTful APIs لتحديد الموارد عبر طرق بروتوكول HTTP القياسية (GET, POST, PUT, DELETE).

4. الـ Middleware (البرمجيات الوسيطة)

التعريف: هو نمط مرشح الأنابيب (Pipeline / Chain of Responsibility Pattern). عبارة عن سلسلة من الفلاتر التي يمر عبرها الطلب والمخرجات.

الشرح: يُستخدم لتحقيق ما يُعرف في هندسة البرمجيات بـ AOP (Aspect-Oriented Programming)، وتحديداً عزل "الاهتمامات المتقاطعة" (Cross-Cutting Concerns) مثل التحقق من الهوية (Authentication)، الصلاحيات (Authorization)، والتشفير، بعيداً عن كود التطبيق الأساسي.

2_4_: دورة حياة الطلب والبيانات (Request Lifecycle & Data Management)

5. الـ Request (كائن طلب الحالة)

التعريف: هو التجسيد البرمجي (Objectification) لبروتوكول نقل النص الفائق (HTTP Request Message).

الشرح: يقوم بتحويل البيانات النصية الخام القادمة من المتصفح عبر الشبكة إلى كائن برمي كبسولي (Encapsulated Object) يحتوي على الميتاداتا (Headers)، الكوكيز، والمدخلات، ويسمح بتطبيق طبقة التحقق من صحة البيانات (Data Validation Layer) قبل وصولها لعمق النظام.

6. الـ Migration (التحكم في إصدارات المخططات)

التعريف: هو نظام التحكم في إصدارات قواعد البيانات (Database Schema Version Control).

الشرح: يحل مشكلة التطور المستمر لقواعد البيانات في المشاريع المشتركة. يتيح تعريف بنية الجداول كأكواد برمجية (Schema-as-Code)، مما يضمن اتساق قاعدة البيانات (Data Consistency) في بيئات التطوير والإنتاج المختلفة، ويوفر ميزة التراجع التبادلي (Deterministic Rollback).

7. الـ Authenticated (حالة إثبات الهوية)

التعريف: هي حالة التحقق من الهوية الرقمية (Identity Verification State).

الشرح: تعني نجاح النظام في ربط الطلب الحالي (HTTP Request) بهوية مستخدم مسجلة في النظام عبر آليات ميكانيكية مثل الجلسات (Sessions) أو الرموز الرقمية (Tokens/JWT)، لكسر طبيعة بروتوكول HTTP عديم الحالة (Stateless).

3_4_: الواجهات والأنظمة المساعدة (UI Components & Subsystems)

8. الـ Component (المكونات النمطية)

التعريف: هو وحدة برمجية ذاتية الاحتواء وقابلة لإعادة الاستخدام (Encapsulated, Reusable UI Unit).

الشرح: يتبع نظرية "التصميم التركيبي" (Component-Based Architecture). يهدف إلى تقليل التكرار البرمجي (DRY Principle - Don't Repeat Yourself) من خلال دمج منطق العرض (Presentation Logic) والتصميم البصري في وحدة واحدة مستقلة.

9. الـ Dashboard (لوحة المراقبة المركزية)

التعريف: هي واجهة تحليل النظم (Analytical Interface / Management Information System - MIS).

الشرح: من الناحية النظرية، تُمثل طبقة التجميع (Aggregation Layer) للبيانات الحيوية، حيث تقوم بتحويل البيانات الخام المعقدة المستخرجة من قنوات متعددة إلى مؤشرات أداء رئيسية (KPIs) ومعلومات بصرية تدعم اتخاذ القرار.

10. الـ Mail (نظام الاتصال الخارجي الموجه)

التعريف: هو الأنظمة الفرعية لإرسال الإخطارات غير المتزامنة (Asynchronous Notification Subsystem).

الشرح: يعتمد على أنماط فصل المهام. يتيح صياغة الرسائل ككائنات برمجية (Mailable Objects) تفصل تصميم الرسالة عن بروتوكول النقل (SMTP, Mailgun)، ويدعم المعالجة غير المتزامنة عبر الطوابير (Queues) لرفع أداء النظام الإجمالي وموثوقيته (System Reliability).

4_4_: التوسع الهيكلي وأنماط الخدمات (Service Architecture)

11. الـ Services (خدمات منطق العمل المعزول)

التعريف: هي طبقة نطاق العمل المعزولة (Domain Service Layer).

الشرح: تُستخدم لتطبيق مبدأ Separation of Concerns (فصل الاهتمامات) ومبادئ SOLID وتحديداً مبدأ المسؤولية الفردية (Single Responsibility Principle). يتم عزل المعالجات الحسابية والتجارية المعقدة خارج الـ Controllers لتصبح مستقلة تماماً عن بروتوكول الويب، مما يسهل إعادة استخدامها في بيئات أخرى (مثل واجهات سطر الأوامر CLI أو الـ APIs).

12. الـ Service Container (حاوية حقن التبعيات)

التعريف: هي حاوية التحكم العكسي (Inversion of Control - IoC Container).

الشرح: هي قلب إطار العمل النابض ونظرياً تُمثل تطبيقاً صارماً لنمط تصميم Dependency Injection (حقن التبعيات). بدلاً من أن يقوم الكلاس بإنشاء الكائنات التي يعتمد عليها يدوياً، تقوم الحاوية بإدارة دورة حياة هذه الكائنات وحقنها تلقائياً عند الحاجة، مما يقلل من الارتباط الوثيق بين الكلاسات (Loose Coupling) ويجعل النظام قابلاً للتوسع والاختبار البرمجي (Unit Testing).

13. الـ Service Providers (مخططات التهيئة المسبقة)

التعريف: هي آلية التمهيد والربط المركزي (Bootstrapping & Binding Mechanism).

الشرح: تُمثل نقطة الانطلاق المركزية (Initialization Vector) لتكوين التطبيق. وظيفتها النظرية هي ربط الخدمات (Services) داخل حاوية الـ IoC وتكوين الإعدادات المشتركة، والإنصات للأحداث (Event Listeners)، بحيث لا يتم استهلاك موارد الخادم لتهيئة أي خدمة إلا عندما يتم استدعاؤها فعلياً (Lazy Loading).

الفصل الثالث

القسم العملي

5_1التصميم الهيكلي والتنفيذي للنظام(Architectural & Implementation Design)

1-هندسة طبقة البيانات (Database Layer Architecture)

في هذا المشروع، تم اعتماد معمارية فصل الاهتمامات (Separation of Concerns)، حيث تم عزل طبقة البيانات بالكامل عن منطق التحكم (Control Logic) واجهة المستخدم. يعتمد النظام على نظام إدارة قواعد البيانات المدمج SQLite بالتكامل مع محرك خرائط الكائنات العلاقية Eloquent ORM المتاح في إطار العمل Laravel.

2-5 مبررات اختيار نظام(SQLite (Architectural Justifications

تم اختيار SQLite كقاعدة بيانات للمشروع بناءً على تحليل المقاييس الهندسيّة التالية:

1. **المعمارية عديمة الخادم :(Serverless Architecture)** على عكس النظم التقليدية) كـ MySQL)، لا يتطلب SQLite خادماً مستقلاً يعمل كعملية خلفية (Background Process)، مما يلغي زمن التأخير الناتجة عن الاتصال عبر الشبكة (Network Overhead)ويجعل النظام ذاتي الاحتواء.(Self-Contained)
2. **التوافقية والاتساق :(ACID Compliance)** يضمن المحرك تحقيق معايير الذرية، الاتساق، العزل، والمتانة($ACID$ $Properties$) ، مما يوفر بيئة آمنة للبيانات تحت مختلف ظروف التشغيل.

3. **سهولة النقل والصيانة :(Portability)** تُخزن قاعدة البيانات بالكامل داخل ملف أحادي الامتداد(SQLite.) ، مما يسهل عملية النسخ الاحتياطي والنقل بين بيئات التطوير والإنتاج دون الحاجة لإعادة ضبط الإعدادات.(Zero-Configuration)

3-5 آلية تصميم وإنشاء المخططات(Database Schema Design via Migrations)

يرتكز تصميم الجداول في النظام على مفهوم **التحكم في إصدارات المخططات (Schema Version Control)**باستخدام الـ .Migrations تتيح هذه الآلية كتابة البنية الهيكلية للجداول بكود برمي كبسولي (Encapsulated PHP Code)يتم ترجمته ديناميكياً بواسطة محرك الإطار إلى استعلامات داتا متوافقة مع قواعد.SQLite

4-5القواعد البرمجية للمخططات(Migration Definitions)

عند تنفيذ أمر الهجرة(Migration) ، يتم استدعاء واجهات التطبيق البرمجية للـ .Schema Builder

5-5العلاقات بين الكائنات(Entity Relationships)

تمت ترجمة القيود العلاقية (Relational Constraints) بين الجداول برمجياً داخل الكلاسات لضمان التكامل المرجعي:(Referential Integrity)

- **نموذج المستخدم :(User Model)** يمثل علاقة "واحد إلى متعدد($One-to-Many$) "، حيث يمكن للمستخدم الواحد امتلاك عدة مهام.

6-5 إدارة واجهة التكوين وبيئة التشغيل (Environment Configuration)

يتم الربط والتعريف الهيكلي بين إطار العمل وقاعدة البيانات عبر الطبقة الوسيطة لـ دافع PDO (PHP Data Objects). يتم عزل متغيرات البيئة داخل ملف التكوين المستقل (.env) لرفع المعايير الأمنية ومن **4.4.1 دورة حياة التنفيذ(Execution Lifecycle)**

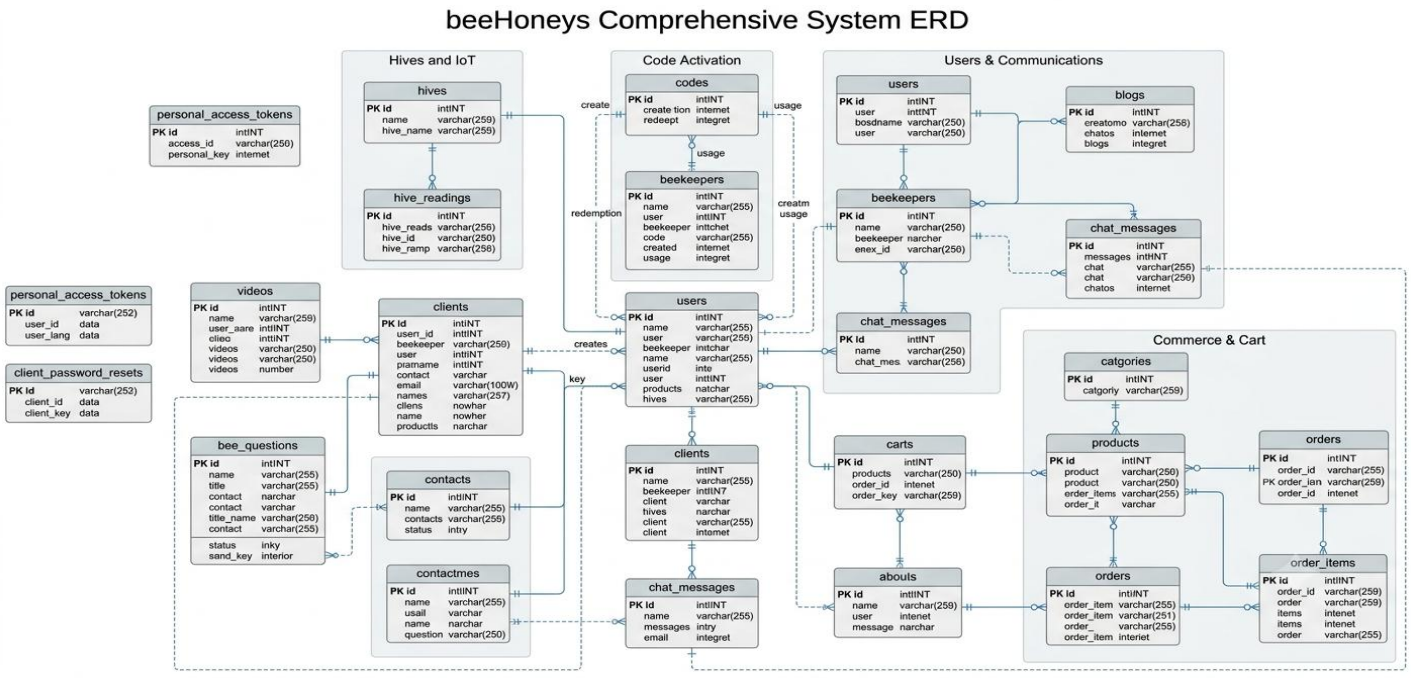
عند بناء واختبار النظام، تمر دورة حياة البيانات بالخطوات المنهجية التالية:

1. **التهيئة البرمجية :**استدعاء الأمر php artisan migrateلقراءة ملفات المخططات وترجمتها إلى لغة DDL (Data Definition Language) متوافقة مع قواعد محرك SQLite المخفف.
2. **التحقق والاتساق :**يقوم المحرك بإنشاء ملف الـ sqlite.وتوليد المخططات المقررة مع تطبيق القيود الهيكلية.(Primary, Unique, Foreign Keys)

3. **المعالجة التشغيلية (Runtime):** عند إرسال أي طلب(Request) ، يقوم الـ Model عبر حزمة Eloquent بفتح مجرى قراءة/كتابة سريع على الملف، ومعالجة البيانات، ثم غلق المجرى تلقائياً، محققاً أعلى معدلات الأداء للنظم المتوسطة والصغيرة.ع صلب البيانات الحساسة (Hardcoding).

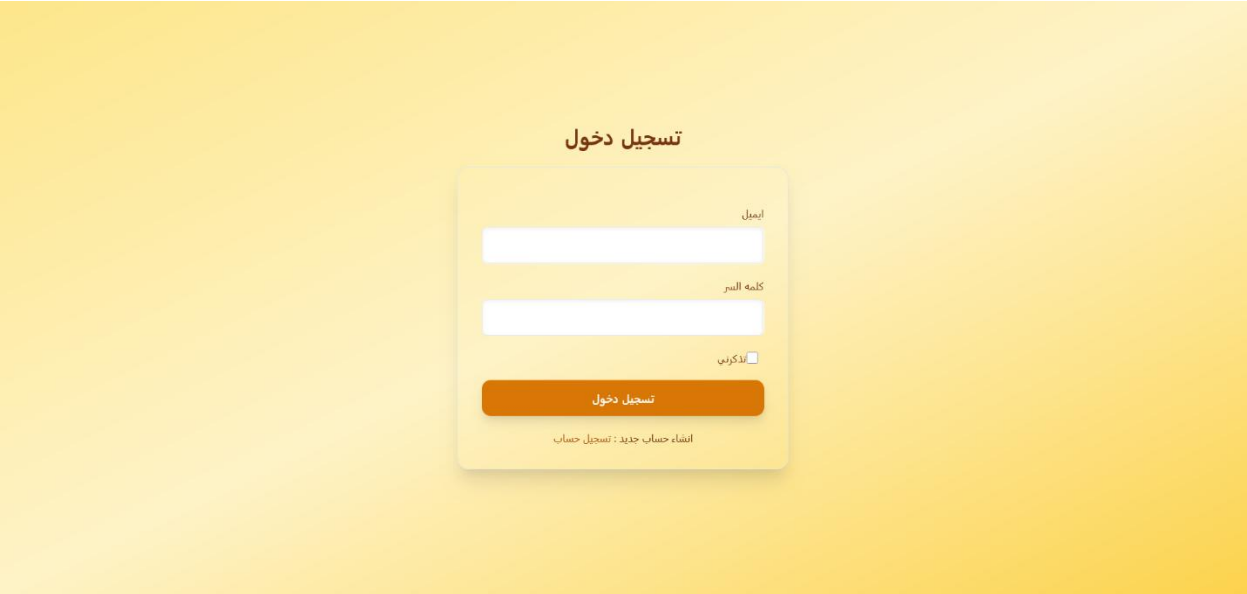


5-7مخطط قاعده البيانات:



6-الواجهات:

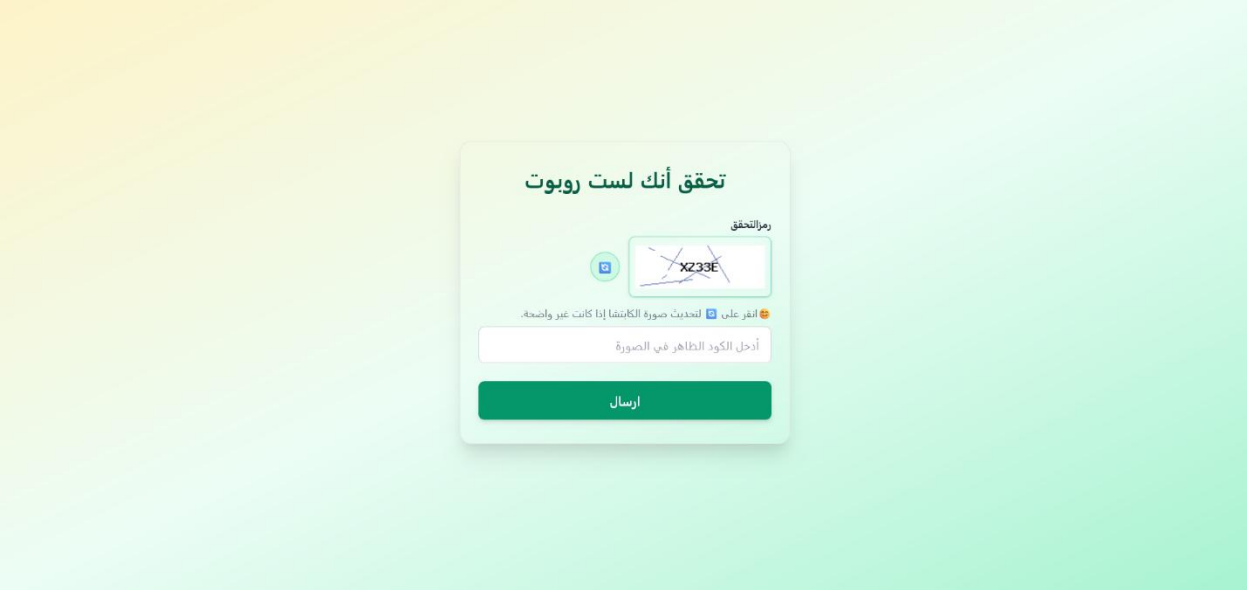
1. واجهة تسجيل الدخول (للنحالين):



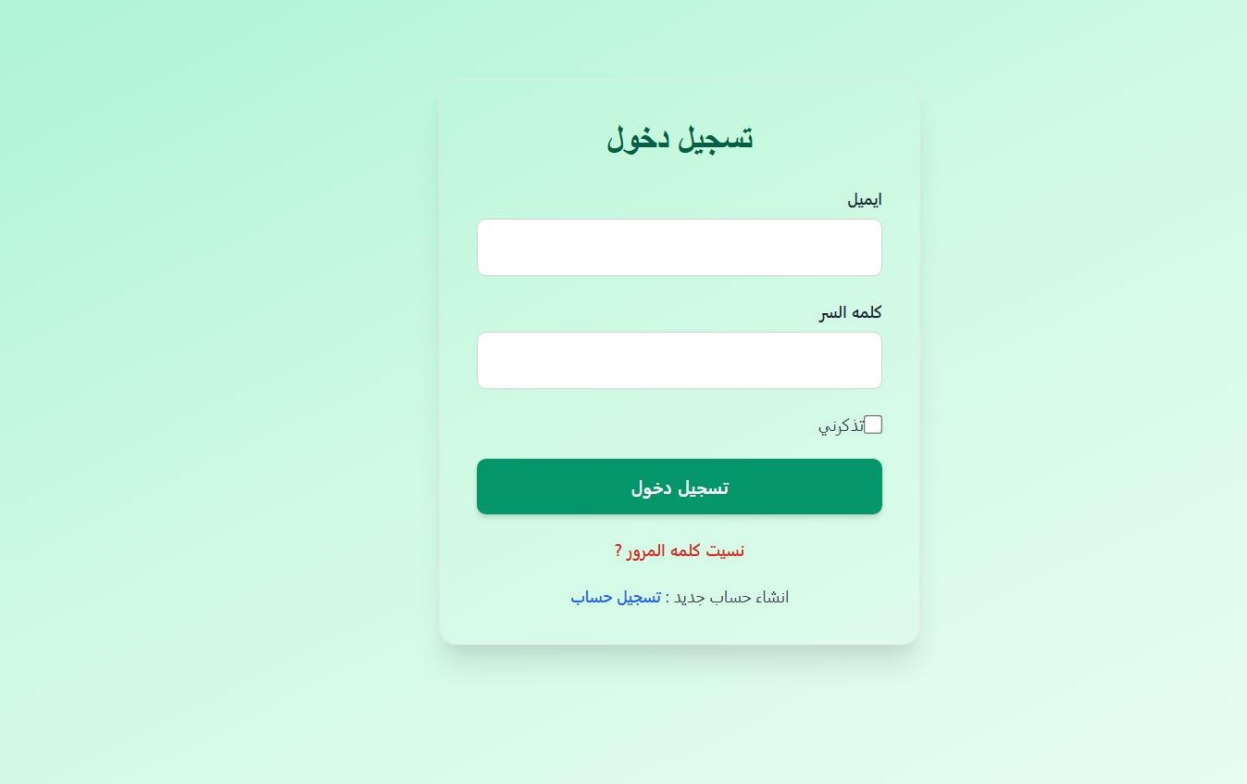
2. واجهة انشاء حساب (للنحالين):



3. واجهة CAPCHA:



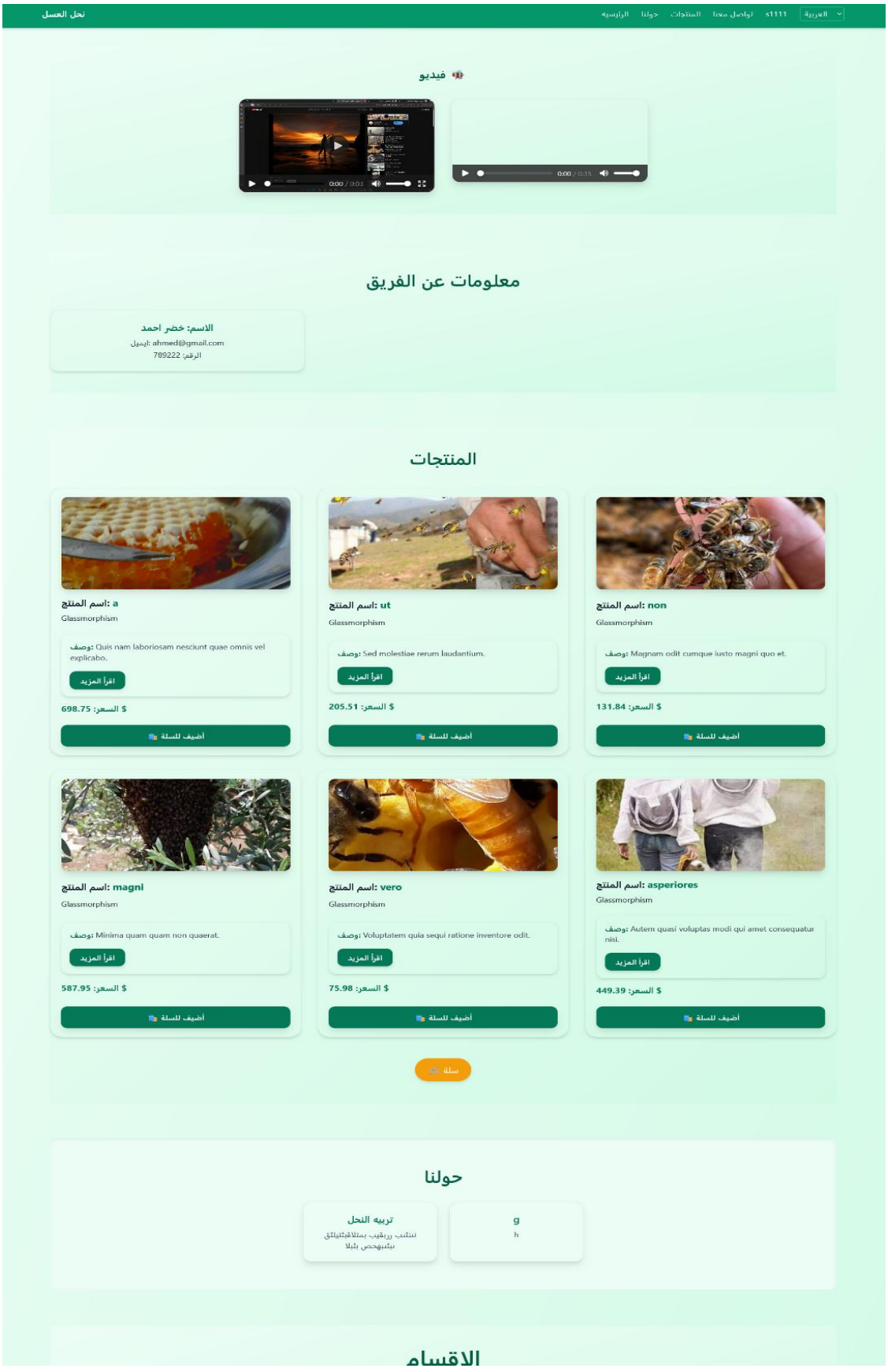
4. واجهة تسجيل الدخول العامة:



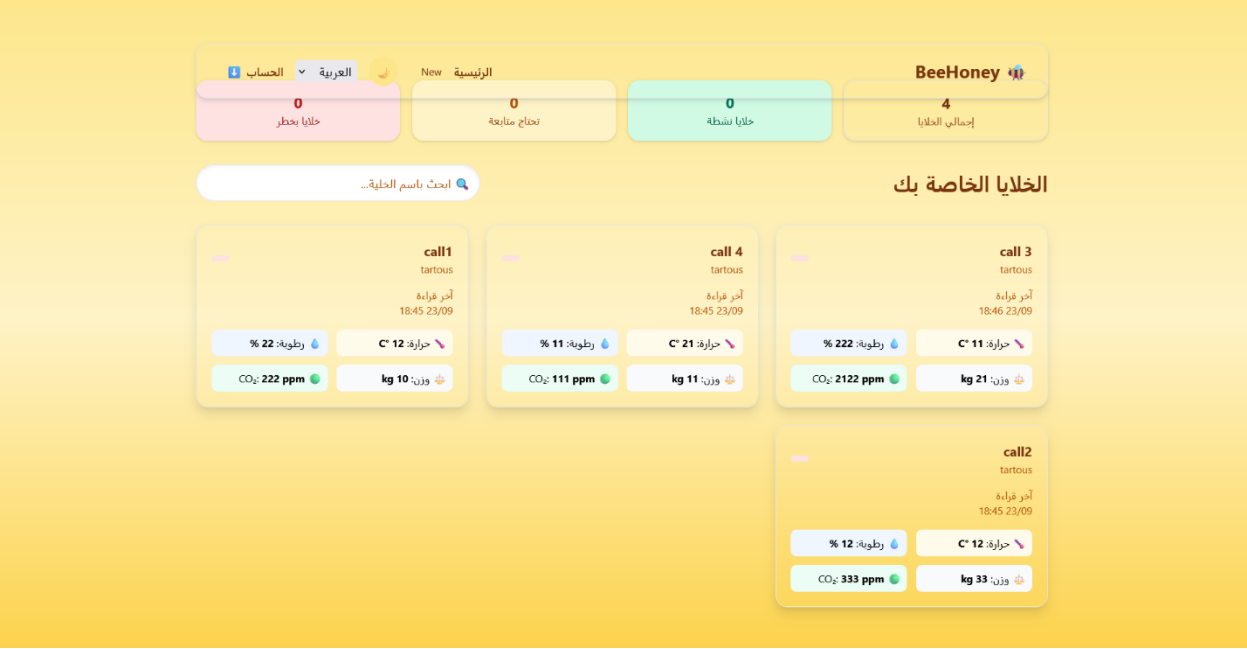
5. واجهة انشاء حساب العامة:



6. الواجهة الرئيسية:


نحل العسل
الرئيسيه
حولنا
المنتجات
تواصل معنا
s1111
العربية
فيديو
0:00 / 0:03
0:00 / 0:35
معلومات عن الفريق
الاسم: خضر احمد
ايميل: ahmed@gmail.com
الرقم: 789222
المنتجات
اسم المنتج: a
Glassmorphism
وصف: Quis nam laboriosam nesciunt quae omnis vel explicabo.
اقرأ المزيد
السعر: 698.75 $
أضيف للسلة
اسم المنتج: ut
Glassmorphism
وصف: Sed molestiae rerum laudantium.
اقرأ المزيد
السعر: 205.51 $
أضيف للسلة
اسم المنتج: non
Glassmorphism
وصف: Magnam odit cumque iusto magni quo et.
اقرأ المزيد
السعر: 131.84 $
أضيف للسلة
اسم المنتج: magni
Glassmorphism
وصف: Minima quam quam non quaerat.
اقرأ المزيد
السعر: 587.95 $
أضيف للسلة
اسم المنتج: vero
Glassmorphism
وصف: Voluptatem quia sequi ratione inventore odit.
اقرأ المزيد
السعر: 75.98 $
أضيف للسلة
اسم المنتج: asperiores
Glassmorphism
وصف: Autem quasi voluptas modi qui amet consequatur nisi.
اقرأ المزيد
السعر: 449.39 $
أضيف للسلة
سلة
حولنا
تربيه النحل
g
h
الاقسام

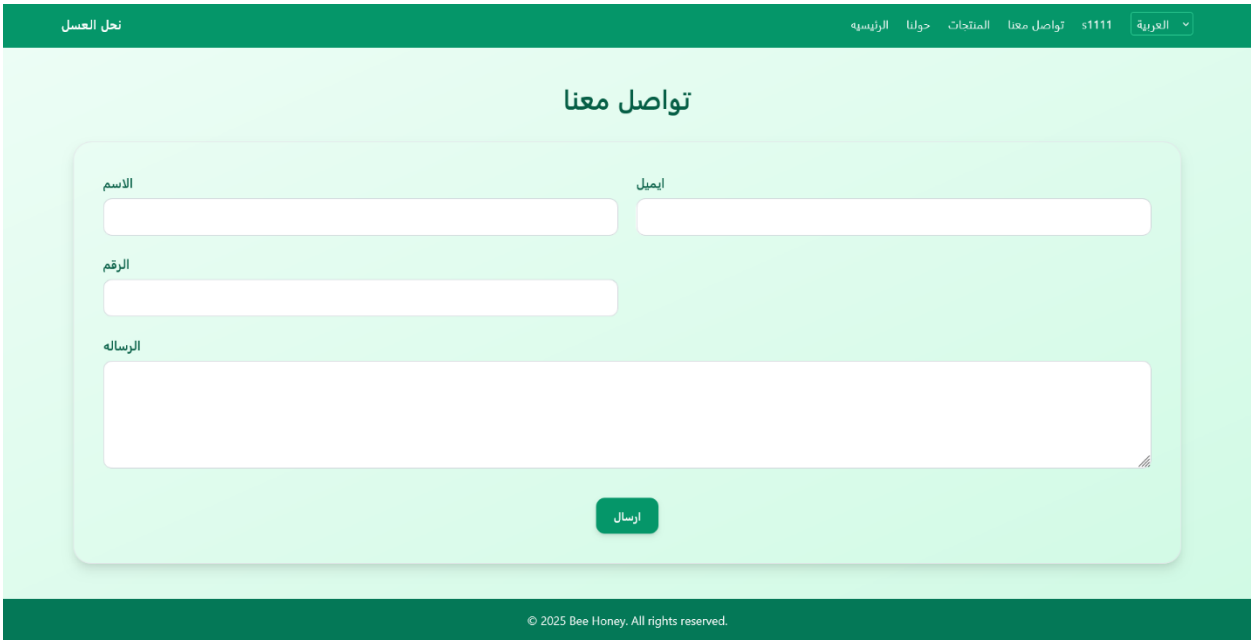
7. واجهة الملف الشخصي للنحال:



8. واجهة محاكاة بيانات الخلية:



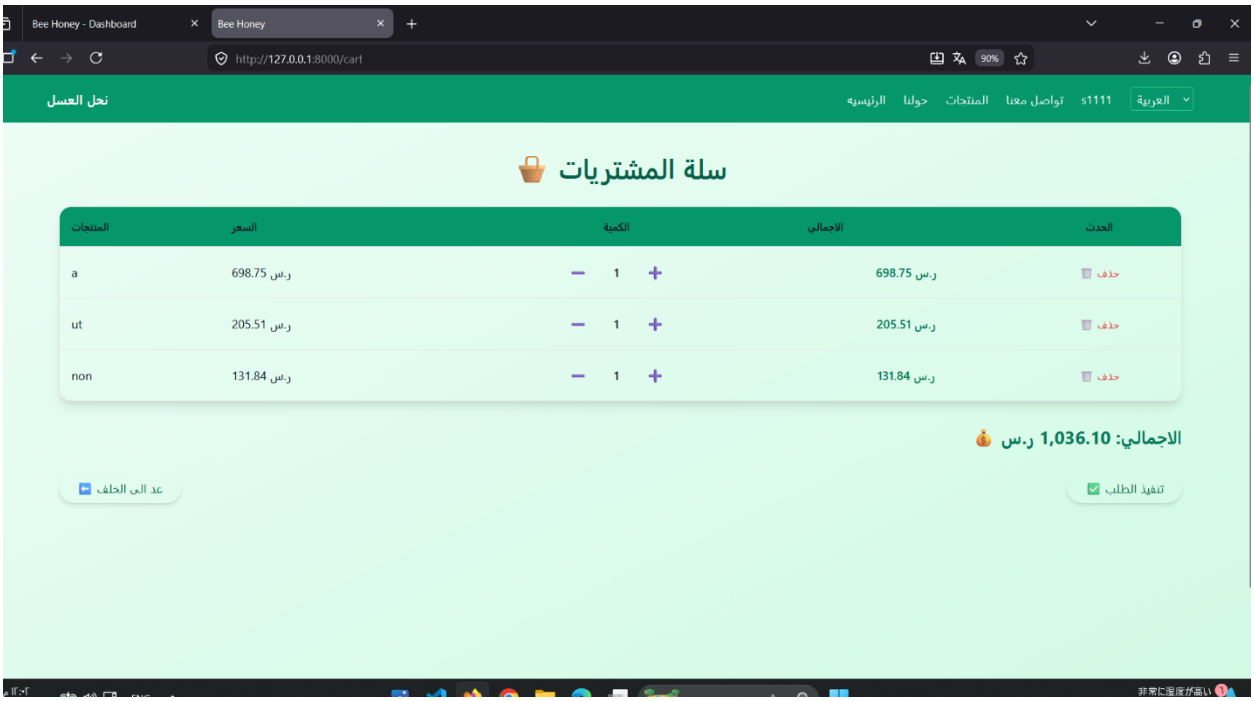
9. واجهة التواصل مع فريق المنصة:



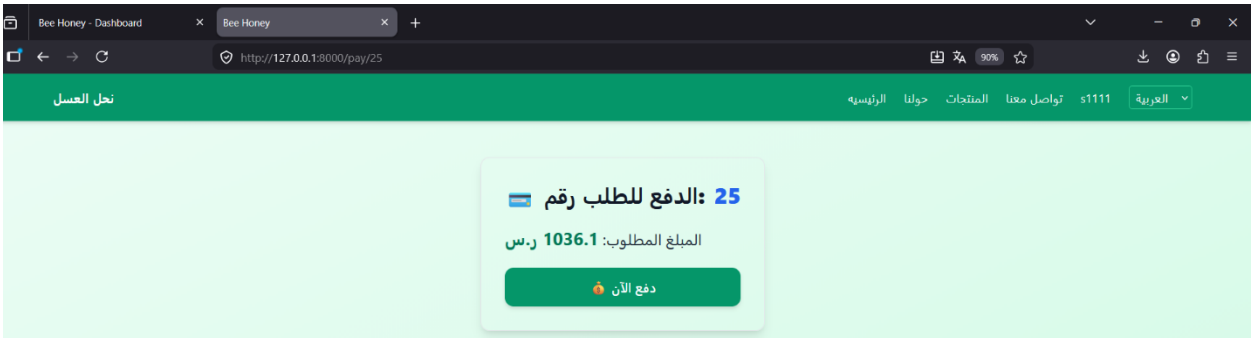
10. واجهة تفقد الفواتير:



11. واجهة تفقد المشتريات:



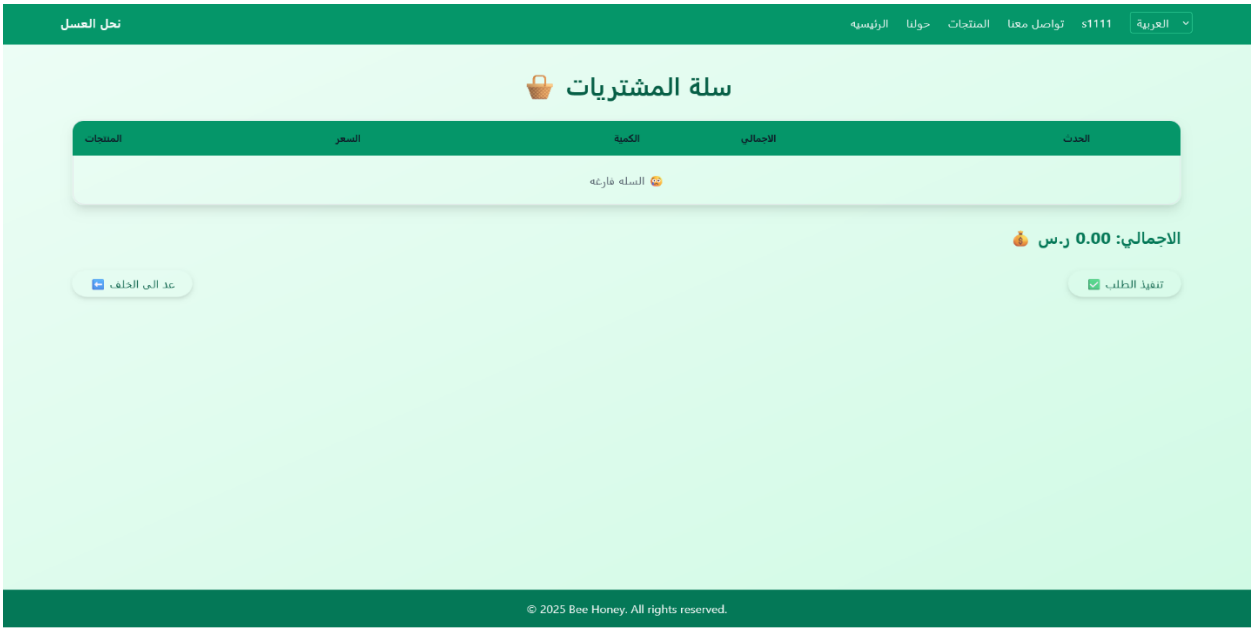
12. واجهة الدفع عند الشراء:



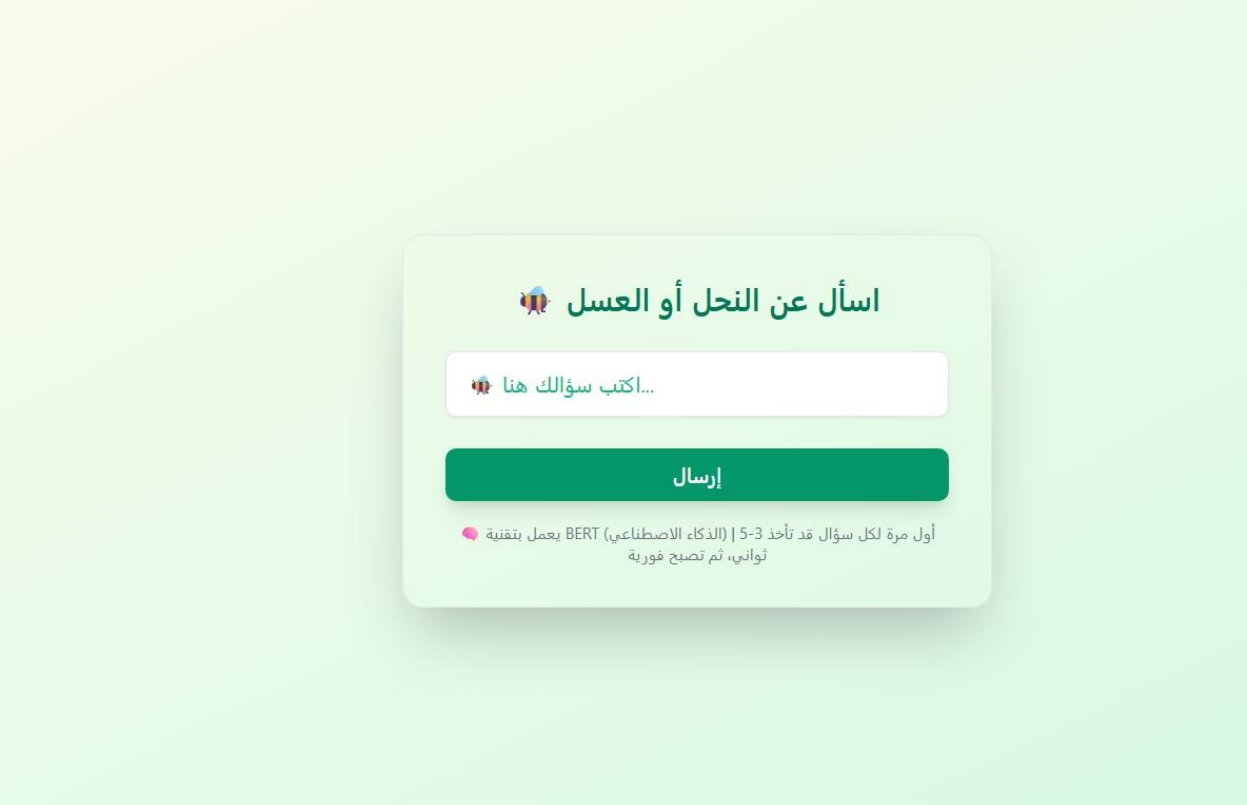
13. واجهة الطلب:



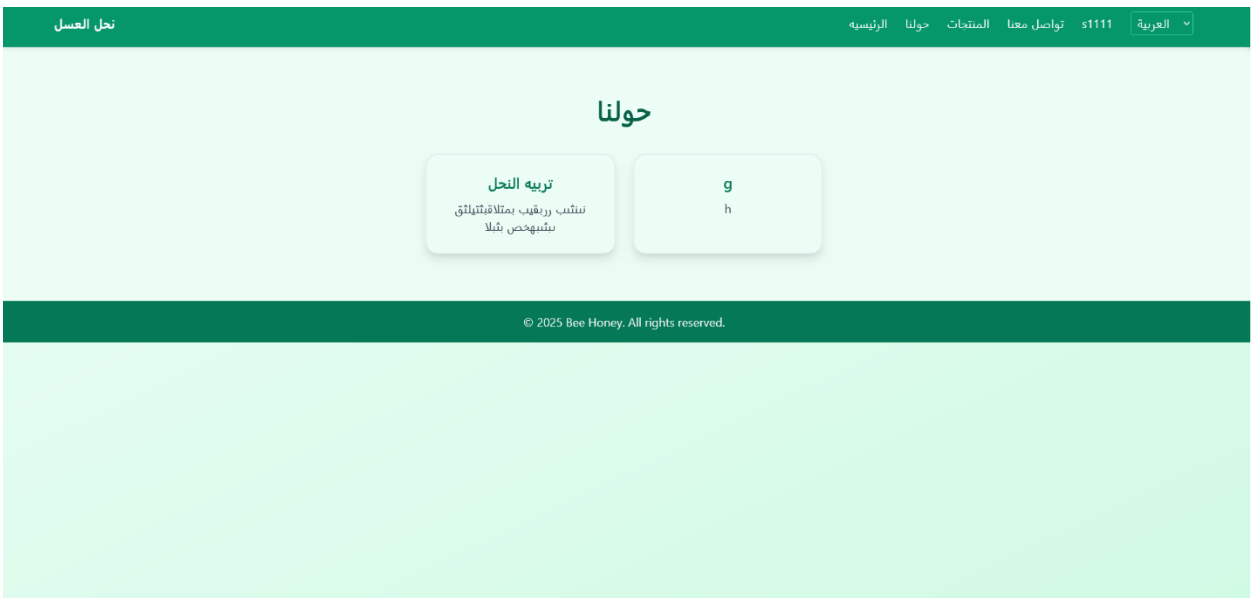
14. واجهة سلة المشتريات:



15. واجهة أسئلة المستخدمين:



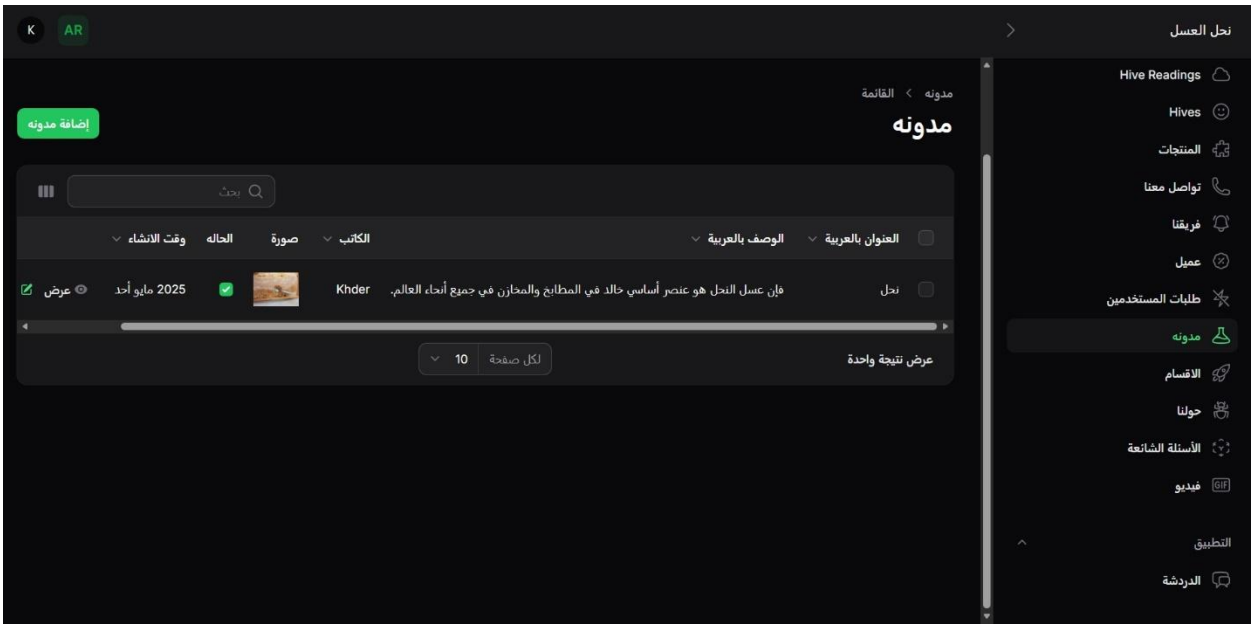
16. حولنا



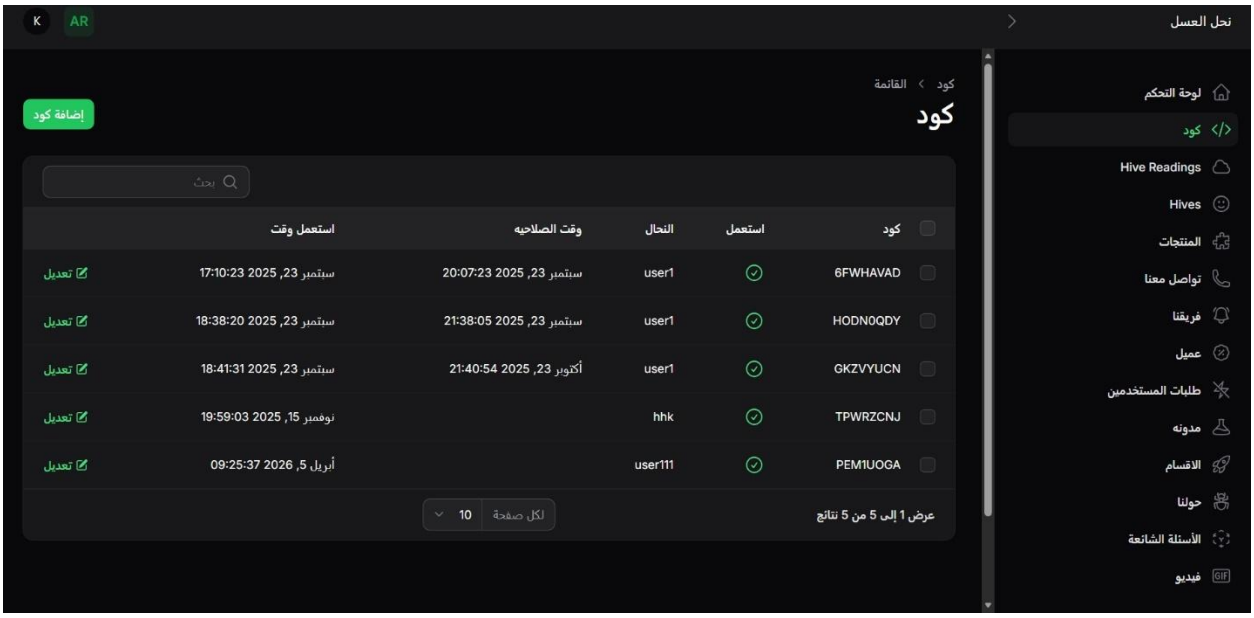
17. واجهة معلومات الفريق:



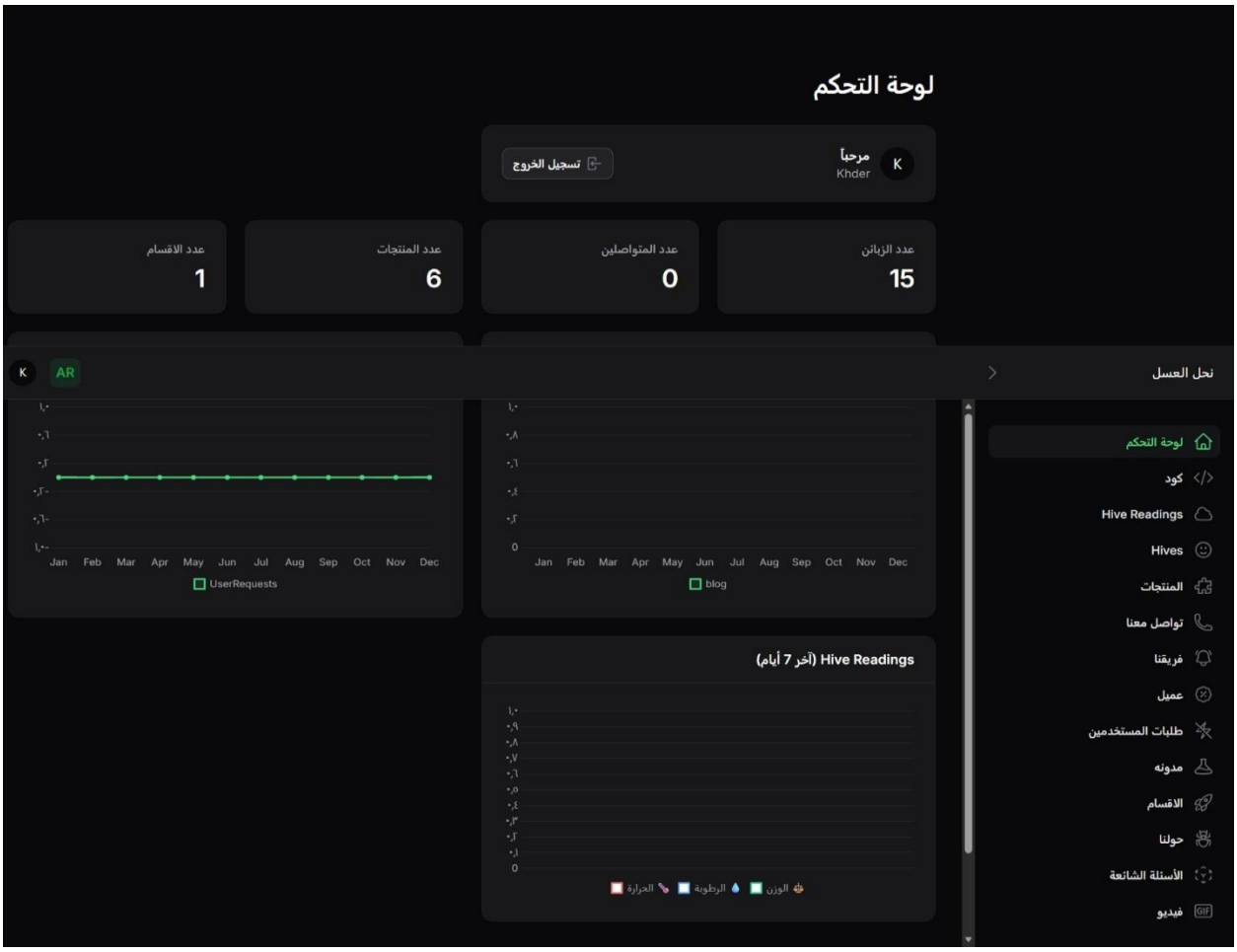
18. واجهة المدونات:



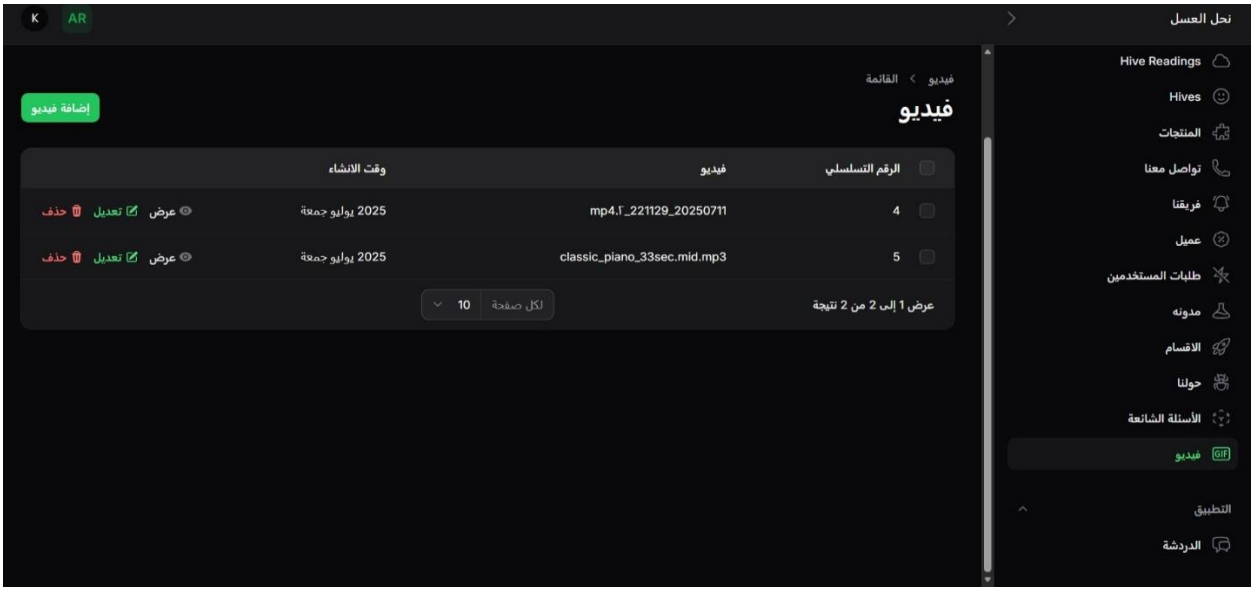
19. واجهة أكواد الشراء:



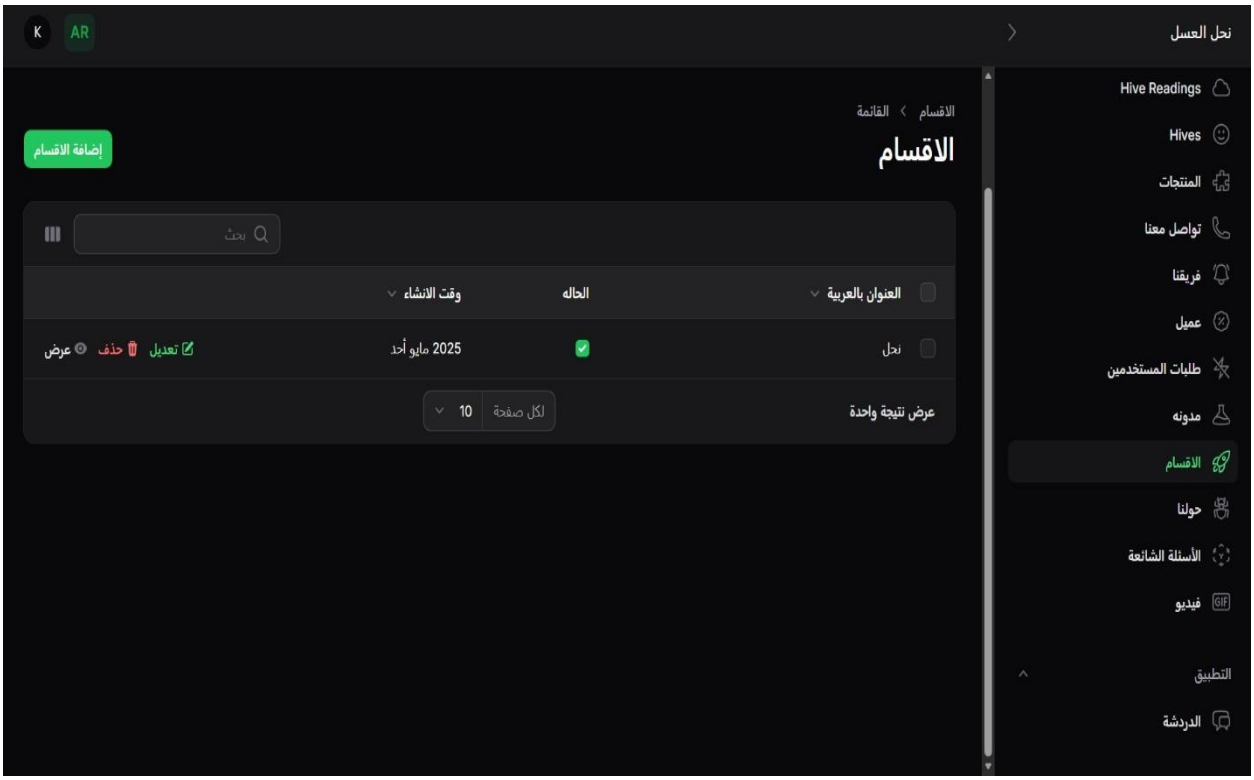
20. واجهة لوحة التحكم:



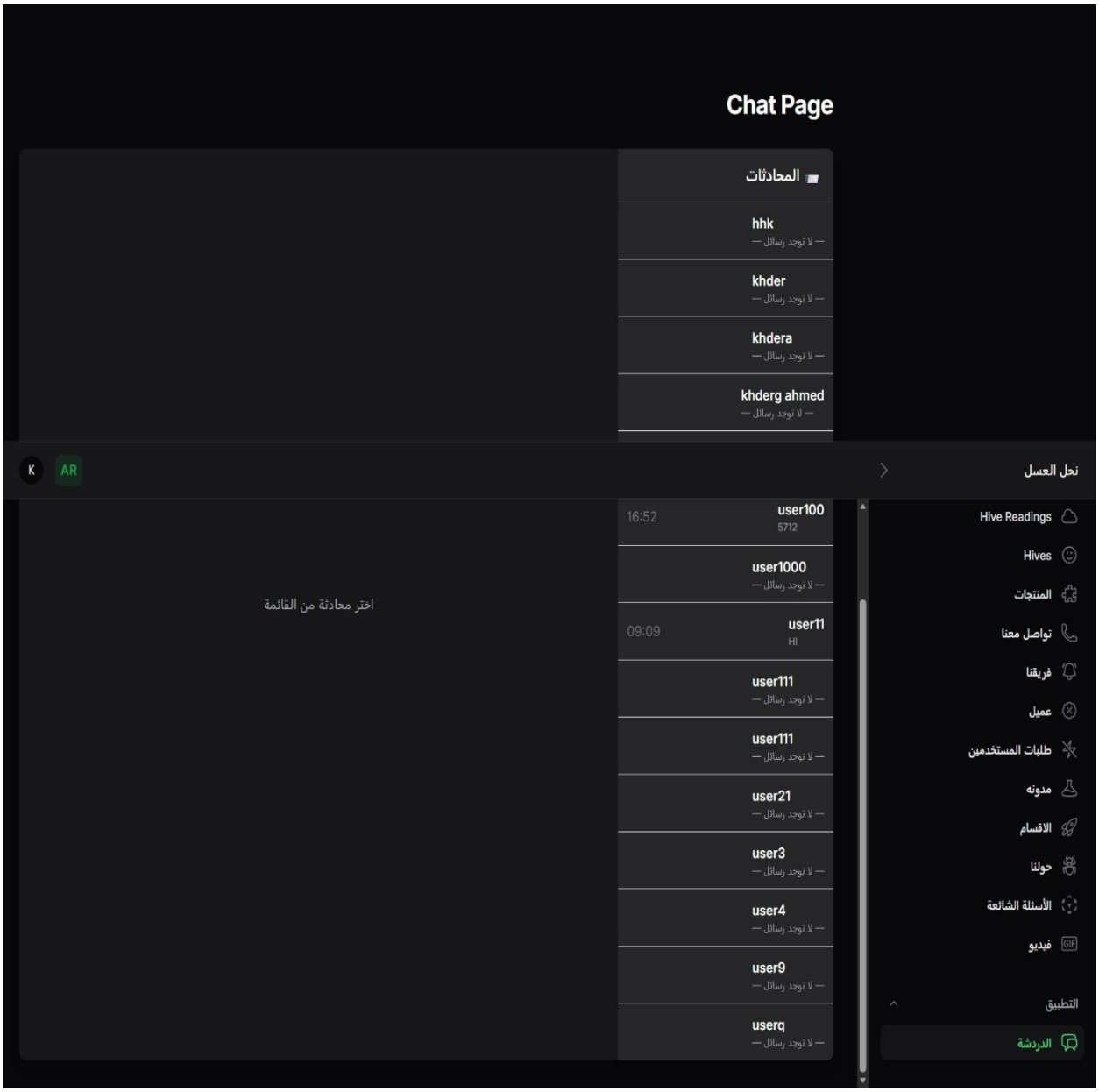
21. واجهة تحميل الفيديوهات:



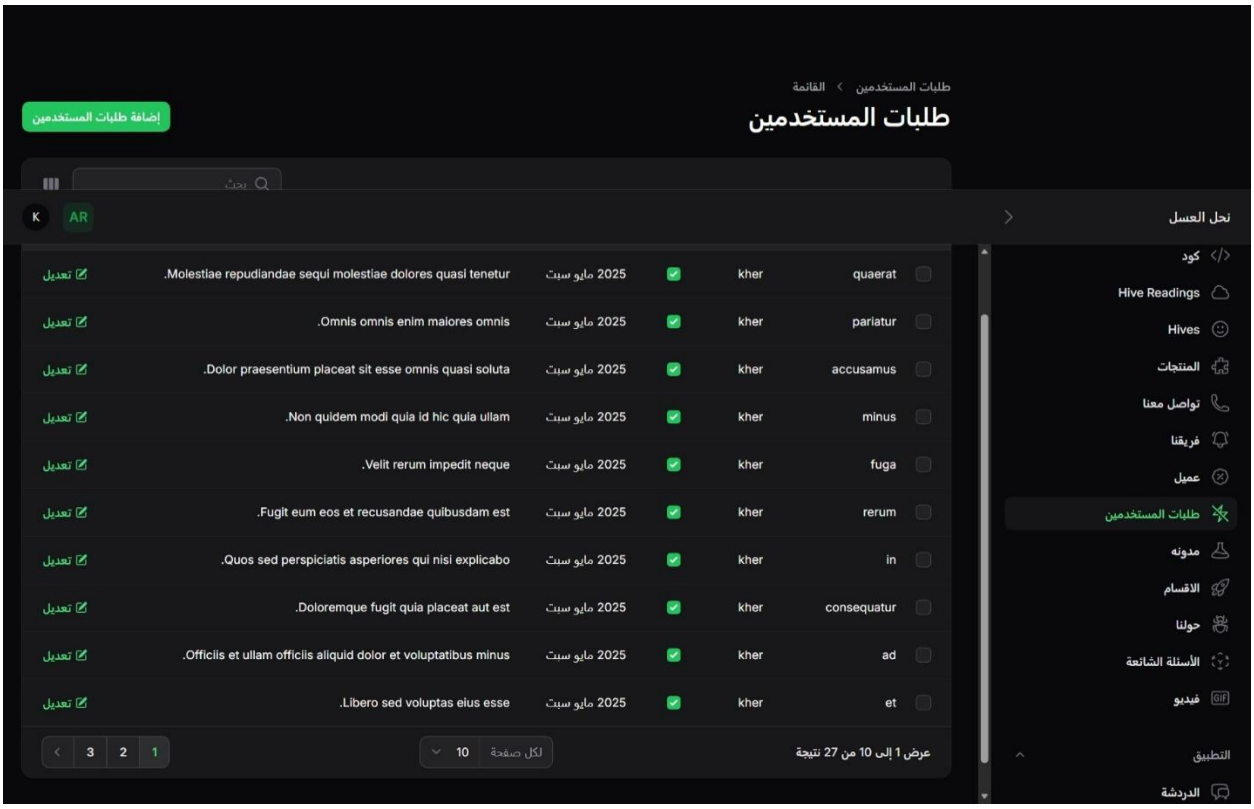
22.واجهة إدارة الأقسام:



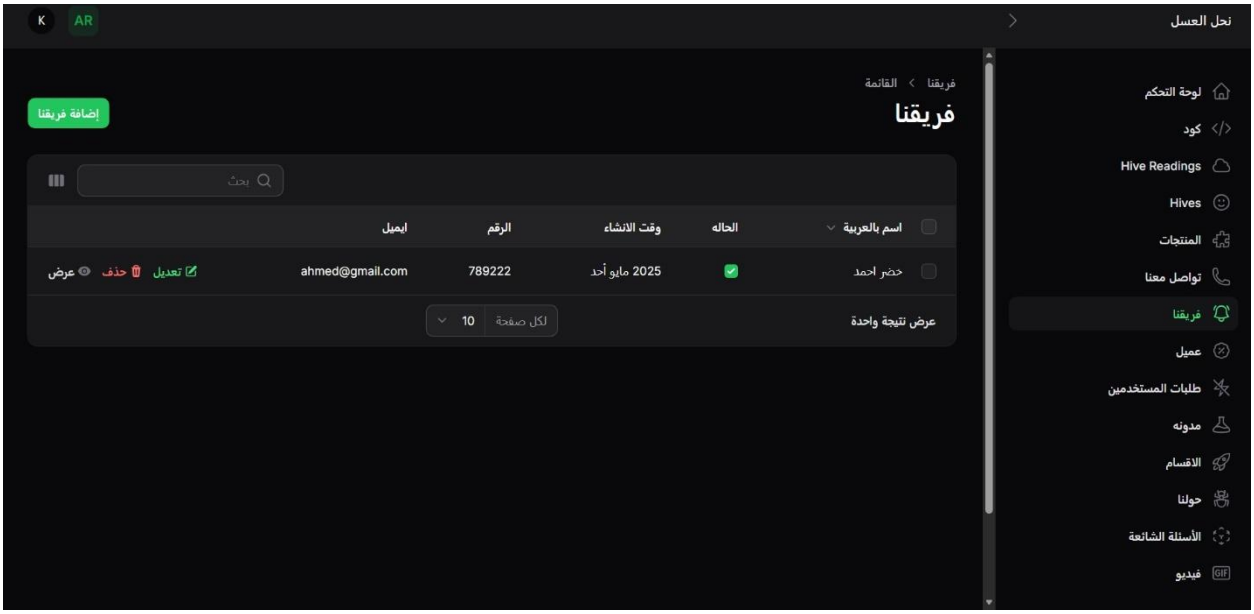
23.واجهة إدارة المحادثات:



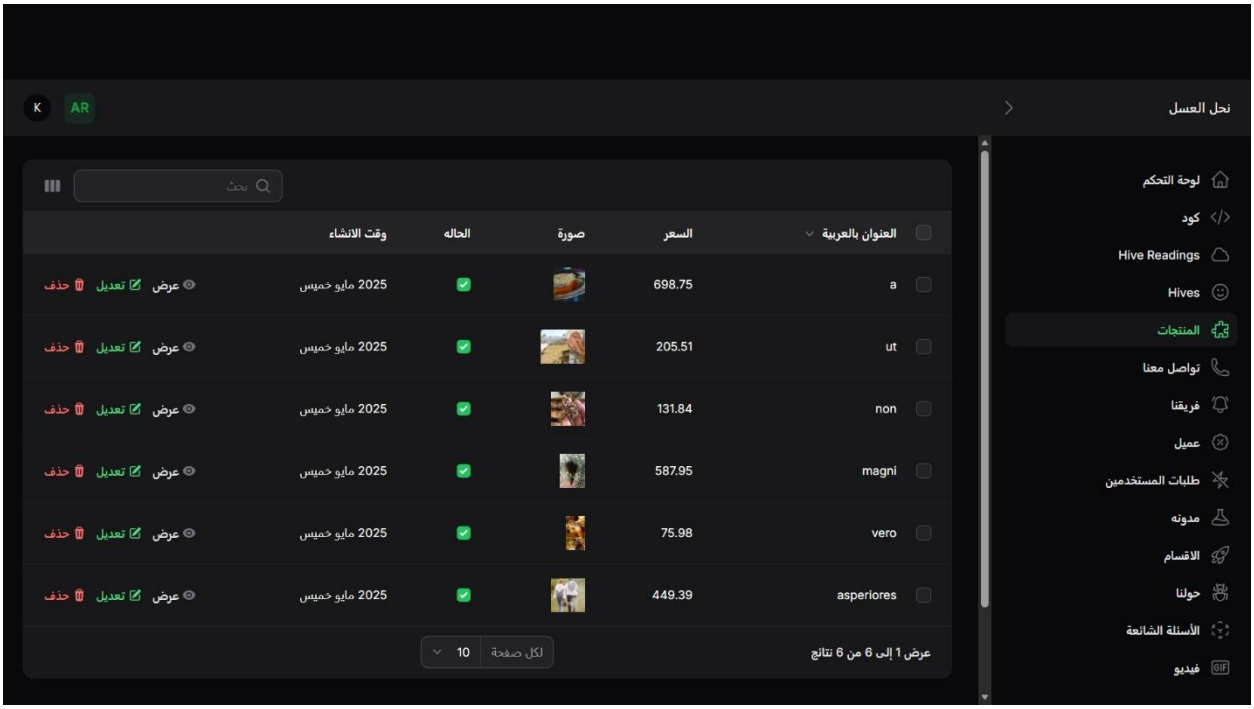
24. واجهة إدارة طلبات المستخدمين:



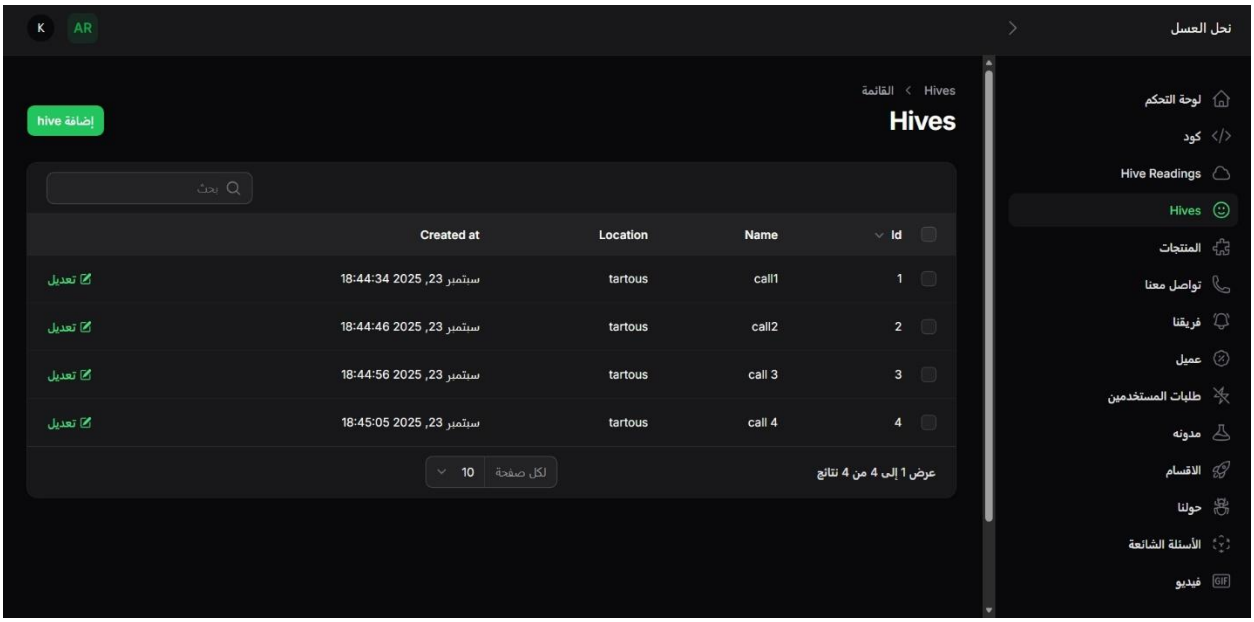
25. الواجهة الخاصة بفريق المنصة:



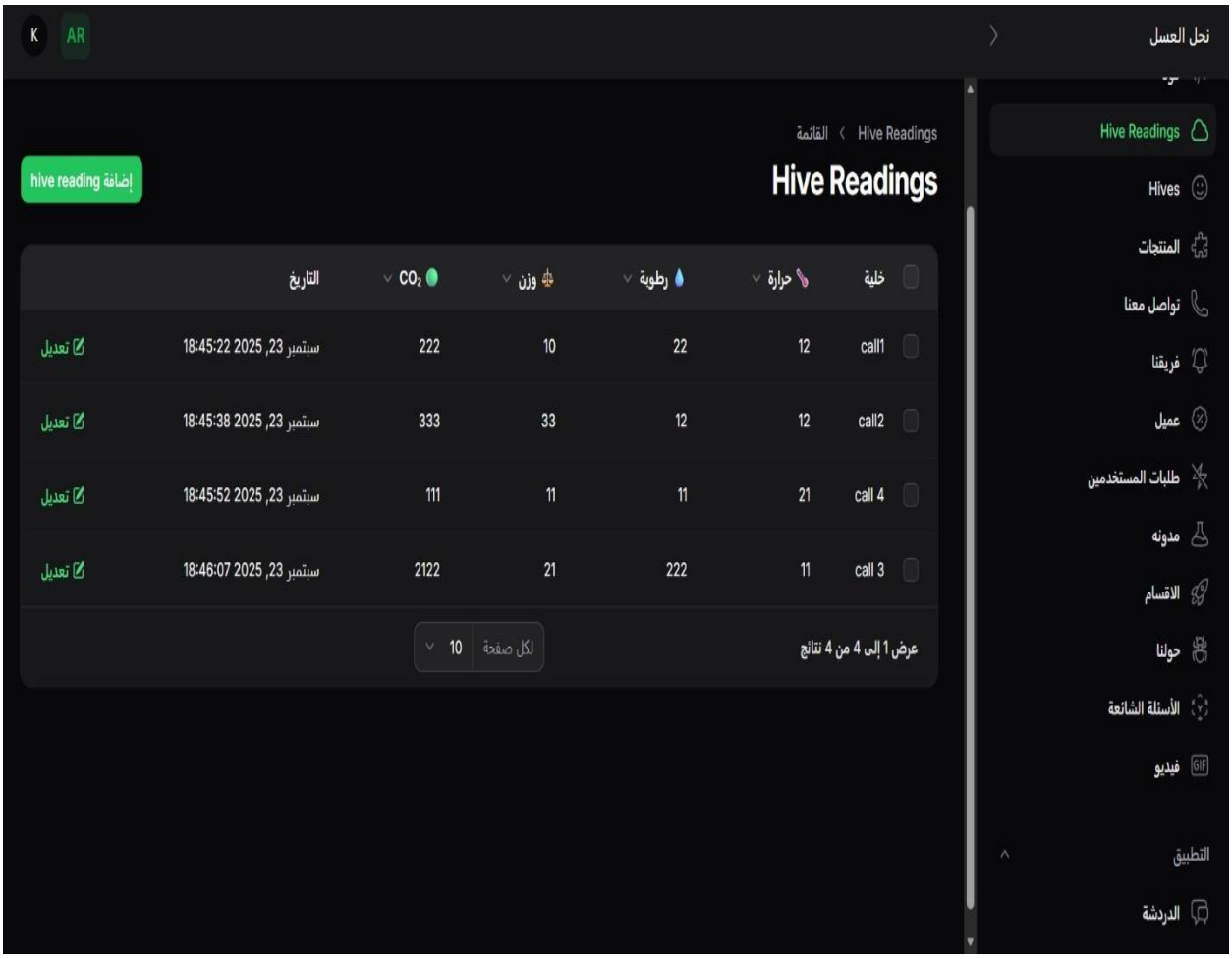
26. واجهة إدارة المنتجات:



27. واجهة إدارة خلايا النحل:



28. واجهة معلومات خلايا النحل:



طريقة التشغيل:

1.مرحلة بناء الاعتمادات الخلفية(Backend Dependencies)

تعتمد بيئة Laravel على مدير الحزم Composerلتركيب المكتبات البرمجية وهيكلة خطوط الربط (Autoloading).يتم ذلك عبر الأمر التالي في الطرفية:(Terminal)

```
composer install
```

- الأثر الأكاديمي :يقوم هذا الأمر بقراءة ملف composer.json وبناء مجلد vendorالذي يحتوي على النواة البرمجية لـ Laravel وحزمة Livewire وكافة المكتبات الوسيطة.

2.

مرحلة بناء اعتمادات الواجهة الأمامية(Frontend Asset Compilation)

بما أنك تستخدم Tailwind CSS، فإن النظام يحتاج إلى محرك Node.jsومدير الحزم NPMلتجميع ملفات التنسيق وفلترتها. يتم التشغيل عبر خطوتين:

- أولاً: تثبيت حزم الواجهة:

```
npm install
```

- ثانياً: تشغيل خادم المراقبة والبناء الفوري:(Vite Compilation Server)

```
npm run dev
```

- يقوم محرك البناء مثل (Vite بمراقبة ملفات Blade وLivewire ، وعبر ميزة-Just-In) (Time CompilerلـTailwind ، يقوم بحقن الخصائص الرسومية (CSS) المستخدمة فعلياً فقط، مما يقلل حجم ملف التنسيق النهائي إلى حده الأدنى، ويضمن التحديث الفوري للواجهات ($Hot\ Module\ Replacement$).

3.مرحلة إعداد البيئة وقاعدة البيانات(Environment & Database Initializing)

- تهيئة ملف البيئة :يتم إنشاء ملف env.وضبط إعدادات الاتصال بقاعدة البيانات) مثل الـ Port واسم القاعدة.(
- توليد مفتاح التشفير الآمن:

```
php artisan key:generate
```

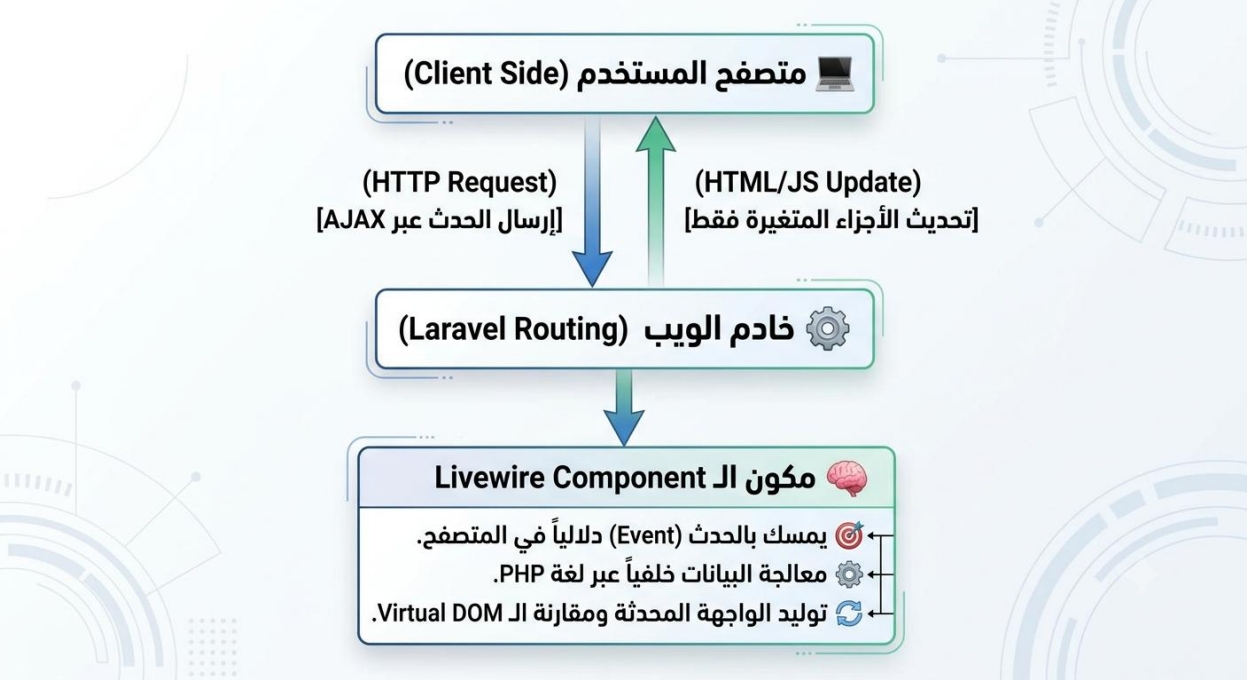
- هجرة الجداول وضخ البيانات:(Migration & Seeding)

```
php artisan migrate --seed
```

4.إطلاق خادم الويب المحلي(Web Server Launch)

أخيراً، يتم تشغيل خادم المحاكاة المحلي لبيئة PHP لاستقبال طلبات الـ:HTTP

```
php artisan serve
```



الفصل الرابع

8-1-التحديات:

التحدي الأول: التعامل مع توليد الكابتشا (CAPTCHA) وربطها بالجهاز عبر الجلسة(Session)

1.البعد الأمني والنظري

تُعد الكابتشا خط الدفاع الأول لحماية الموارد الخلفية للنظام ($Backend\ Resources$) من هجمات الحرمان من الخدمة ($DoS/DDoS$) والطلبات المؤتمتة الخبيثة ($Bot\ Attacks$) التي قد تحاول إغراق محرك الذكاء الاصطناعي (BERT) بأسئلة عشوائية مستمرة لاستهلاك موارد الخادم.

2.آلية الربط وهندسة الجلسات(Session-Device Binding)

المعضلة الهندسية هنا تكمن في خاصية "عدم حفظ الحالة ($Stateless$) "لبروتوكول .HTTP لحل هذا التحدي، يعتمد النظام على المبادئ التالية:

- توليد الرمز العشوائي :عند طلب الواجهة، يقوم الخادم بتوليد رمز عشوائي مشفر (Token) وتحويله ديناميكياً إلى صورة مرئية) عبر مكتبات معالجة الصور مثل GD أو Imagick في PHP)يصعب على برامج القراءة الآلية (OCR) تمييزها.
- الربط التلازمي :(Co-relation) يتم تخزين القيمة النصية الصافية للرمز داخل جلسة المستخدم الحالية (Session) المشفرة على الخادم، والتي ترتبط بجهاز المستخدم عبر ملف تعريف ارتباط آمن ($Session\ Cookie$) يحمل معرفاً فريداً.(Session ID)
- مرحلة المطابقة :(Verification Phase) عند إرسال النموذج، يقارن النظام المدخل القادم من جهاز المستخدم مع القيمة المحفوظة في الجلسة المربوطة بالمعرّف الفريد. إذا تطابقتا، يتم تمرير الطلب إلى معالج الذكاء الاصطناعي، وإلا يتم تدمير الجلسة ومطالبة المستخدم بإعادة التحقق، مما يضمن أن الطلب نابع من تفاعل بشري حقيقي ومن نفس الجهاز الذي طلب الواجهة.

🎨 التحدي الثاني: تحسين تجربة المستخدم بالتصميم المتجاوب عبرTailwind CSS

1.البعد الهندسي للتصميم المتجاوب(Responsive Architecture)

بما أن مشروعك مخصص لمجال حيوي (مثل إدارة النحل والعسل)، فمن المتوقع أن يتم تصفحه من بيئات عمل مختلفة: هواتف ذكية في الميدان (المناحل)، أو شاشات لوحية(Tablets) ، أو حواسب مكتبية في الإدارة.

2.استراتيجية الحل باستخدامTailwind CSS

يعتمد إطار العمل Tailwind على فلسفة "الهاتف أولاً(Mobile-First Philosophy) "، وتم حل التحدي عبر الآليات التالية:

- نقاط التوقف الديناميكية :(Breakpoints) يتم بناء الواجهة افتراضياً لتعمل على أصغر الشاشات، ثم يتم استخدام البادئات الذرية (مثل :mdللشاشات المتوسطة، و :lgللشاشات الكبيرة) لإعادة توزيع العناصر بصرياً.
- المرونة الهيكلية :(Flexbox & Grid Systems) بدلاً من استخدام قياسات ثابتة بالبكسل تتلف عند تغيير حجم الشاشة، تم اعتماد شبكات العرض المرنة (grid-cols-1 md:grid-cols-3) والتي تضمن ترتيب عناصر لوحة التحكم وإحصائيات المناحل بشكل عمودي مريح على الهواتف، وأفقي متكامل على الشاشات الكبيرة.
- تحسين الأداء البصري :استخدام تأثيرات الزجاج الخلفي (backdrop-blur) والبطاقات الأنيقة المناسبة للمس على الهواتف، لضمان تجربة مستخدم سلسة دون التأثير على سرعة تحميل أصول الواجهة الجرافيكية.($Asset\ Loading$)

التحدي الثالث: التأكد من توافق النظام مع المتصفحات المختلفة(Cross-Browser Compatibility)

1.طبيعة التحدي(The Compatibility Dilemma)

تختلف محركات رندرة وتفسير الأكواد بين المتصفحات العالمية الكبرى (مثل محرك Blinkفي Chromeو Edge ، ومحرك Geckoفي Firefox ، ومحرك WebKitفي Safari التابع لشركة Apple).عدم التوافق قد يؤدي إلى تشوه الخطوط العربية، أو فشل بعض مؤثراتTailwind ، أو تجمد واجهات Livewire التفاعلية.

2.الاستراتيجية الهندسية لضمان التوافق

تمت معالجة هذا التحدي وعلاجه عبر ثلاثة مستويات تقنية:

- المستوى الأول: معالجة الـ CSS التلقائية:(Autoprefixing)

أثناء عملية تجميع ملفات Tailwind عبر محرك البناء(Vite/Webpack) ، يتم استخدام أداة أكاديمية تسمى PostCSSمصحوبة بـ .Autoprefixerتقوم هذه الأداة بقراءة الخصائص الحديثة (مثل التظليل أو الأبعاد المرنة) وحقن السوابق الخاصة بالمتصفحات تلقائياً (مثل -webkit-لمتصفح سفاري و -moz-لمتصفح فايرفوكس) لضمان ظهور التصميم المتدرج والجميل بنفس الدقة على جميع المنصات.

- المستوى الثاني: توحيد البنية الأساسية) مبدأ الـ:(Normalization

تتضمن حزمة Tailwind ميزة مدمجة تسمى) Preflightوهي نسخة مطورة من حزم Normalize.css).تقوم هذه الميزة بمسح كافة التنسيقات الافتراضية المتباينة التي تفرضها المتصفحات تلقائياً على العناصر (مثل الهوامش والخطوط الافتراضية)، وتصفيرها لتبدأ جميع المتصفحات من نقطة انطلاق موحدة ومحايدة تماماً.

- المستوى الثالث: استقرار البرمجيات التفاعلية:(JavaScript/Livewire Polyfills)

يعتمد Livewire في تبادل البيانات على طلبات الـ Fetch ووعود الـ JavaScript الاستباقية. تم التأكد من توافقية هذه السكربتات مع المعايير الحديثة للمنظمة العالمية($W3C$) ، لضمان أن عمليات التحديث الفوري للواجهة ومحاكاة الـ Virtual DOM تعمل بكفاءة مستقرة دون أي انهيار برمي في المتصفحات التي تطبق قيود حماية صارمة مثل متصفح Safari على أنظمة.iOS

8-2-توصيات :

أولاً: التوصيات الخاصة بالبنية التحتية والهندسة البرمجية (Architectural & Software Engineering)

- الانتقال إلى المعمارية الخدمية المنفصلة :(Microservices Architecture) يوصى بفصل محرك معالجة اللغات الطبيعية (بايثون) عن إطار عمل الويب الرئيسي(Laravel) ، وبنائه كخدمة مستقلة ومستضافة بذاتها عبر إطارات عمل مخصصة مثل .(FastAPI) يهدف هذا

الإجراء إلى منع استهلاك موارد الخادم الرئيسي أثناء العمليات الحسابية المعقدة وضمان موازنة الأحمال العالية.($Load\ Balancing$)

- تبني تقنية المعالجة غير المتزامنة :(Asynchronous Process Management) يوصى بإدراج نظام الطوابير والمهمات المؤجلة (Laravel Queues) مدعوماً بـ (Redis) لإدارة طلبات المستخدمين الموجهة للذكاء الاصطناعي، مما يضمن استقرار استجابة الخادم وتجنب تجمد واجهة العرض أثناء معالجة النصوص الطويلة.
- ترقية نظام الذاكرة المؤقتة :(Advanced Caching Optimization) التوصية بربط الذاكرة المؤقتة للنظام بقواعد بيانات في الذاكرة الصرفة (In-Memory Databases) لتسريع عمليات جلب وتدقيق الأسئلة المتكررة، وتقليل معدل استدعاء بيئة نظام التشغيل الأساسية ($Shell\ Execution$).

ثانياً: التوصيات الخاصة بمحرك الذكاء الاصطناعي ومعالجة البيانات(AI & NLP Optimizations)

- الضبط الدقيق المخصص للنموذج :(Domain-Specific Fine-Tuning) يوصى بإعادة تدريب أو ضبط الأوزان الرياضية (Fine-Tuning) لنموذج المعالجة اللغوية المعتمد (BERT/MiniLM)باستخدام حزمة بيانات (Corpus) محلية ومتخصصة بدقة في مفردات ومصطلحات [اذكر مجال مشروعك هنا، مثل: تربية النحل وإنتاج العسل]، وذلك لرفع دقة حساب التشابه الجيبي ($Cosine\ Similarity$) إلى حدها الأقصى.
- أتمتة حلقة التعلم المستمر :(Continuous Learning Automation) التوصية بتطوير واجهة برمجية مخصصة للمشرفين (Admin Dashboard) تتيح تصفية الأسئلة التي سجلت مؤشرات تشابه منخفضة (أقل من العتبة المحددة 0.3)، بهدف تصحيح إجاباتها بشرياً وإعادة ضخها تلقائياً لتحديث قاعدة البيانات المرجعية.(dataset.json)
- التحول إلى قواعد البيانات الدلالية العلاقية :التوصية بنقل البيانات المعرفية من ملفات JSON الثابتة إلى جداول علاقاتية مؤشّرة ($Indexed\ Relational\ Databases$) لضمان سلامة البيانات وسرعة الاستعلام مع تضخم حجم المدخلات المعرفية مستقبلاً.

ثالثاً: التوصيات الخاصة بتجربة المستخدم وتفاعل الواجهات (UI/UX & Human-Computer Interaction)

- دعم تقنيات التعرف على الصوت :(Speech-to-Text Integration) يوصى بدمج واجهات برمجية صوتية تمكّن المستخدمين (خاصة النحالين أو العاملين في البيئات الميدانية) من طرح استفساراتهم صوتياً دون الحاجة للكتابة النصية، مما يرفع من مستوى سهولة الاستخدام وفلسفة الوصول الشامل.($Accessibility$)

- تطوير واجهة محادثة حية تفاعلية :(Real-time Conversational UI) التوصية بتحويل واجهة إدخال البيانات الحالية إلى بيئة محادثة حية شبيهة بأنظمة المراسلة الفورية الاستباقية، وذلك بالاستفادة الكاملة من ميزات التحديث الفوري اللحظي التي يوفرها إطار العمل.(Livewire)

8-3-خاتمة:

مع تسارع الخطى في عصر التحول الرقمي والذكاء الاصطناعي، لم تعد الأنظمة البرمجية مجرد أدوات لأتمتة المهام التقليدية، بل غدت عقولاً تكنولوجية ممتدة تُسهم في دعم اتخاذ القرار وتطوير القطاعات الحيوية. ويأتي هذا المشروع، المتمثل في تطوير [نظام "Works Bee" لإدارة المناحل والذكاء الاصطناعي]، ليكون نموذجاً تطبيقياً يجسد التناغم الفني والهندسي بين أحدث أطر تطوير الويب وتقنيات معالجة اللغات الطبيعية (NLP).لقد كان الهدف الجوهري لهذا العمل هو كسر الفجوة التقليدية بين بيئات الويب الديناميكية ومحركات الحوسبة العصبية المعقدة. ومن خلال تبني معمارية برمجية متكاملة تدمج بين مرونة وقوة إطار العمل Laravel، والتفاعل اللحظي المبتكر لبيئة Livewire، والانسيابية البصرية لإطار Tailwind CSS، نجح النظام في تقديم تجربة مستخدم متجاورة ومتوافقة سيبرانياً مع مختلف البيئات التشغيلية.ولم يتوقف الابتكار عند حدود الإدارة الجافة للبيانات، بل تعداها إلى إضفاء طابع معرفي ذكي عبر دمج نموذج الفضاء المتجهي ثنائي الاتجاه BERT في بيئة Python. هذا التكامل سمح للنظام بالانتقال من الفهم الحرفي الجامد لأسئلة المستخدمين إلى الفهم الدلالي والسياقي العميق، مدعوماً بآليات معالجة وحماية صارمة مثل هندسة الجلسات المتزامنة وتأمين المقابس البرمجية، بالإضافة إلى ترشيد استهلاك موارد الخادم عبر أنظمة الكاش الذكية.إن العقبات التقنية التي واجهت مسيرة بناء هذا النظام— بدءاً من إدارة الجلسات وتأمين الـ Captcha، وصولاً إلى تحديات التوافقية بين المتصفحات وعزل البيئات التشغيلية المتزامنة—لم تكن إلا محطات صقلت المتانة الهندسية للمشروع، وأثبتت جدوى الفلسفات البرمجية الحديثة القائمة على الفصل المقترن الضعيف ($Loose\ Coupling$).ختاماً، لا يمثل هذا المشروع نهاية المطاف، بل هو حجر أساس ونقطة انطلاق واعدة نحو جيل جديد من الأنظمة الهجينة الذكية. ونأمل أن يفتح هذا العمل آفاقاً جديدة للمطورين والباحثين في كيفية طوع التكنولوجيا العصبية لخدمة التطبيقات التطبيقية الميدانية، ومواكبة تطلعات الرؤى التكنولوجية الحديثة نحو بناء مجتمعات معرفية رقمية ومستدامة.

المراجع

1- إطار العمل:Laravel

Otwell, T. (2026). *Laravel: The PHP framework for web artisans* **(**Version 11.x). Laravel Foundation. **https://laravel.com/docs**

2-محرك الواجهات:Livewire

Reinink, C. (2026). *Livewire: Dynamic interfaces for Laravel* (Version 3.x). Livewire Core Team**.** **https://livewire.laravel.com/docs**

3- إطار التنسيق:Tailwind CSS

Wathan, A. (2026). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. Tailwind Labs Release**.** **https://tailwindcss.com/docs**

4- حزمة لوحات التحكم:FilamentPHP

Thaden, S. (2026). *Filament: A collection of beautiful full-stack components for Laravel administration* (Version 3.x). FilamentPHP. **https://filamentphp.com/docs**

5- لغة البرمجة:PHP

The PHP Development Team. (2026). *PHP: Hypertext Preprocessor manual*. PHP Documentation Group**.** **https://www.php.net/docs.php**

6- لغة البرمجة :Python

Python Software Foundation. (2026). *Python language reference* (Version 3.12). Python Software Foundation. **https://docs.python.org/3/**

7- مجتمع المطورين:Stack Overflow

Stack Exchange Inc. (2026). *Stack Overflow: The open community for anyone that codes*. Stack Exchange Network. **https://stackoverflow.com**

8- منصة المقالات التقنية:Medium

Medium Media. (2026). *Medium: Where good ideas find you (Data Science and Deep Learning sections)*. A Medium Corporation. **https://medium.com**